

Problem Solving Strategies and Computational Approaches SCS1304

Handout 8 : Branch Bound

Prasad Wimalaratne PhD

Branch and Bound

A systematic method for solving optimization problems

1. Goal : Find the optimal solution

Unlike backtracking, which focuses on feasibility (any solution meeting the constraints), branch and bound searches for the best solution among many options.

2. Branch : Divide the problem

Split the problem into smaller subproblems by exploring the different choices available at each step.

3. Bound: Eliminate suboptimal paths

Apply logical tests or bounds to discard branches that cannot produce a better result, speeding up the search.

Branch and Bound

- Branch and Bound is similar to Backtracking(last lecture), but differs!
 - **Backtracking** uses **DFS** to generate state sapce tree
 - Backtracking can be used to find **max or min** , but inefficient as all need to be found, then can take max or min ; hence **avoid Backtracking for optimization problems**
 - Branch and Bound **only generate branches** which will **lead to a fruitful solution** and NOT generate all branches like Backtracking
- The concept branch and bound and backtracking **follow the Brute force method and generate the state space tree.**
- But both of them **follows different approaches.**

Backtracking vs B&B

	Backtracking	Branch and Bound
Target	Find <i>a feasible solution or all</i> solutions	Find the <i>optimal</i> solution
Applications	<i>Constraint</i> satisfaction (e.g., N-Queens, Sudoku, subset sum)	<i>Optimization</i> (e.g., knapsack, TSP, job assignment)
Pruning basis	<i>Feasibility only</i> (i.e. cut a branch if it violates a constraint)	<i>Feasibility and a bound</i> (i.e. cut a branch if it <i>ca not beat</i> the current best solution)
Traversal Method(s)	Almost always DFS (recursion/LIFO stack)	Flexible (DFS, BFS, or Least-Cost (best-first) search)

Branch Bound

- Branch and Bound explores possible solutions systematically, but uses **bounds to avoid exploring branches that cannot lead to an optimal solution.**
- **Branch and Bound in Optimization Problems**
 - Many optimization problems in computer science involve finding the **best solution from a finite or countable set of possibilities.**
 - **Some problems**, such as finding a shortest path or minimum spanning tree, can be solved efficiently **using polynomial-time algorithms.**
 - However, many combinatorial optimization problems may require examining **a very large number of possible combinations** or permutations.

Branch and Bound reduces this search by:

- Branching:** Dividing the problem into smaller subproblems or paths.
- Bounding:** Calculating a bound on the best solution possible from each branch.
- Pruning:** Discarding branches that cannot produce a better solution than the best one found so far.

Different Search Techniques in Branch and Bound

The Branch and Bound algorithm incorporates different search techniques to traverse a state space tree.

1. LC Search (**Least-Cost Search**) : Always expands the node with the **most promising bound (lowest cost or best estimate) first**, regardless of its position in the tree. This is a best-first exploration strategy.
2. BFS (**Breadth-First Search**) : Explores the state space tree **level by level**, expanding **all nodes at the current depth** before moving to the next level.
3. DFS (**Depth-First Search**) : Dives down **one branch as far as possible before backtracking**, typically managed using a stack (LIFO) structure.

Branch Bound : search techniques

- 1. LC search (Least Cost Search):
 - It uses a **heuristic cost function** to compute the bound values at each **node**. Nodes are added to the list of live nodes as soon as they get generated.
 - The node with the **least value of a cost function** selected as a next **E-node** (Expansion Node). (i.e E-node is Live node currently being expanded.)
- 2. BFS (Breadth First Search):
 - It is also known as a **FIFO search**.
 - It maintains the list of live nodes in first-in-first-out order i.e, in a **queue**, The **live nodes are searched in the FIFO order** to make them next E-nodes.
- 3. DFS (Depth First Search):
 - It is also known as a **LIFO search**.
 - It maintains the **list of live nodes in last-in-first-out order** i.e. **in a stack**.
 - The live nodes are searched in the LIFO order to make them next E-nodes.

When to Apply the Branch and Bound Algorithm?

Branch and Bound is appropriate when;

- The problem is a discrete optimization problem.
- The **goal is to find the best solution**, such as minimum cost, maximum profit, or shortest path from a finite or countable set of possible solutions.
- The **decision variables take discrete values** rather than continuous values.
- It is particularly useful for **combinatorial optimization problems**, where there may be a very large number of possible combinations.
- The algorithm systematically explores **possible solutions while using bounds to eliminate non-promising branches**.

Basic Concepts of Branch and Bound

1. **State Space Tree**: Branch and Bound uses a state space tree to **represent and systematically explore the solution space** of a problem.
2. **E-node (Expansion Node)**: The E-node is the live node that is **currently being expanded**.
3. **Generating Children**: When an E-node is expanded, all of its children are generated before another live node is selected for expansion.
4. **Status of E-node**: The current E-node remains an E-node while its children are being generated.
 - Once the E-node has NO further promising branches to explore, it becomes a **dead node**.

In **Backtracking**, nodes are generally explored **depth-first**, whereas **Branch and Bound** selects the next live node based on a strategy such as the best bound.

Branch Bound: Bounding function

- The bounding function helps Branch and Bound **decide which paths are worth exploring** and which paths **can be safely discarded**.
- A **bounding function** is used in the Branch and Bound algorithm to **reduce the search space and find an optimal solution** efficiently.
 - It evaluates the **lower and/or upper bound** of possible solutions at each node.
 - The bound indicates **how promising a partial solution is**.
 - These bounds help determine which branches may lead to an optimal solution.
 - If a node can NOT produce a solution better than the best solution found so far, that node is **pruned without further exploration**.
 - The algorithm then explores **other promising branches**.
 - **The best feasible solution found so far** is taken as the solution to the problem

Branch Bound: When to Prune

- Prune if the bound is NOT promising, the solution is infeasible, OR the node represents a completed solution that does NOT improve the current best.
- The three reasons for pruning a search path at the current node in a Branch and Bound algorithm is as follows:
 1. Bound is not promising

If the node's bound can not improve the best solution found so far, the node is discarded.

 - Maximization: $\text{bound} \leq \text{current best value}$
 - Minimization: $\text{bound} \geq \text{current best value}$
 2. Infeasible solution

If the node violates one or more problem constraints, it is discarded because it cannot lead to a valid solution.
 3. Complete feasible solution

If the node represents a complete feasible solution (a single point), compare it with the current best solution.

 - If it is better, update the best solution.
 - Otherwise, discard it.

Solution Representation Methods

- The solution of the B&B problem can be represented in **two** ways:
 1. **Variable size solution**: Using this solution, we can find the subset of the given set that gives the optimized solution to the given problem.
 - For example, if we have to select a combination of elements from {A, B, C, D} that optimizes the given problem, and it is found that A and B together give the best solution, then the solution will be {A, B}.
 2. **Fixed-size solution**: There are 0s and 1s in this solution, with the **digit at the i^{th} position** indicating whether the i^{th} element should be **included**, for the above example, the solution will be given by {1, 1, 0, 0}, here **1** represent that we have **select** the element which at i^{th} **position** and **0** represent we **do NOT select** the element at i^{th} **position**.

Solution Representation Methods..

- Example :
- Jobs = {j1, j2, j3, j4}
- Assume j1 and j2 together give the best solution
- The solutions can be written in two ways, as shown below:
- If we want to do jobs j1 and j2, the solution can be represented :
 1. Variable Size Solution
 - The subsets of jobs are the first way to represent the solutions.
 - $S1 = \{j1, j2\}$,
 2. Fixed Size Solution
 - The second way to represent the solution is that the first and second jobs are completed, and the third and the fourth jobs are NOT completed .
 - $S2 = \{1, 1, 0, 0\}$,

Branch Bound: Classification

- **Classification** of Branch and Bound **Problems**:
- The Branch and Bound method can be classified into **three types** based **on the order in which the state space tree is searched.**
 1. FIFO Branch and Bound
 2. LIFO Branch and Bound
 3. Least Cost-Branch and Bound
- To denote the solutions in these methods, we will use the **variable solution method**. i.e {A,B} etc not {1,1,0,0} as describe before

Branch and Bound Problem Types

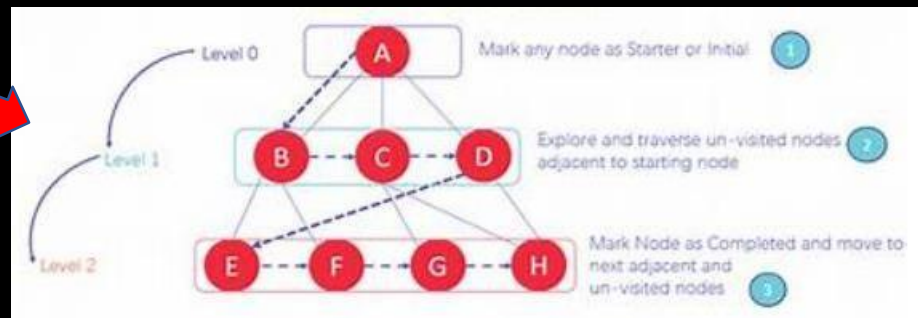
B&B can be classified based on the *data structure* used to manage live nodes, or based on the *search paradigm* name.

Name	Underlying data structure	Equivalent search technique
FIFO Branch and Bound	Queue (First-In-First-Out)	BFS expands all nodes at one level before moving to the next
LIFO Branch and Bound	Stack (Last-In-First-Out)	DFS dives deep down one branch before backtracking
Least Cost Branch and Bound	Priority queue (ordered by cost or bound)	LC Search always expands the most promising node next

Branch Bound: FIFO →

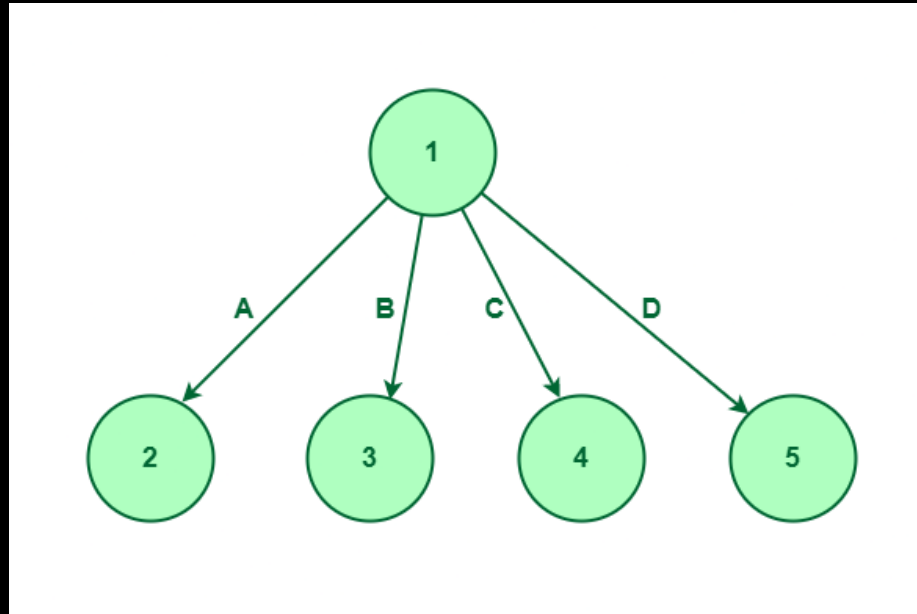


- 1. FIFO Branch and Bound
- **First-In-First-Out** is an approach to the branch and bound problem that uses the queue approach to create a state-space tree.
- In this case, the **breadth-first search (BFS)** is performed, that is, the elements at a **certain level are all searched**, and then the **elements at the next level are searched**, starting with the first child of the first node at the previous level.
- FIFO Branch and Bound uses a **Queue** to **explore the next node** in **BFS fashion**



Branch Bound: FIFO

- For a given set {A, B, C, D}, the state space tree will be constructed as follows :



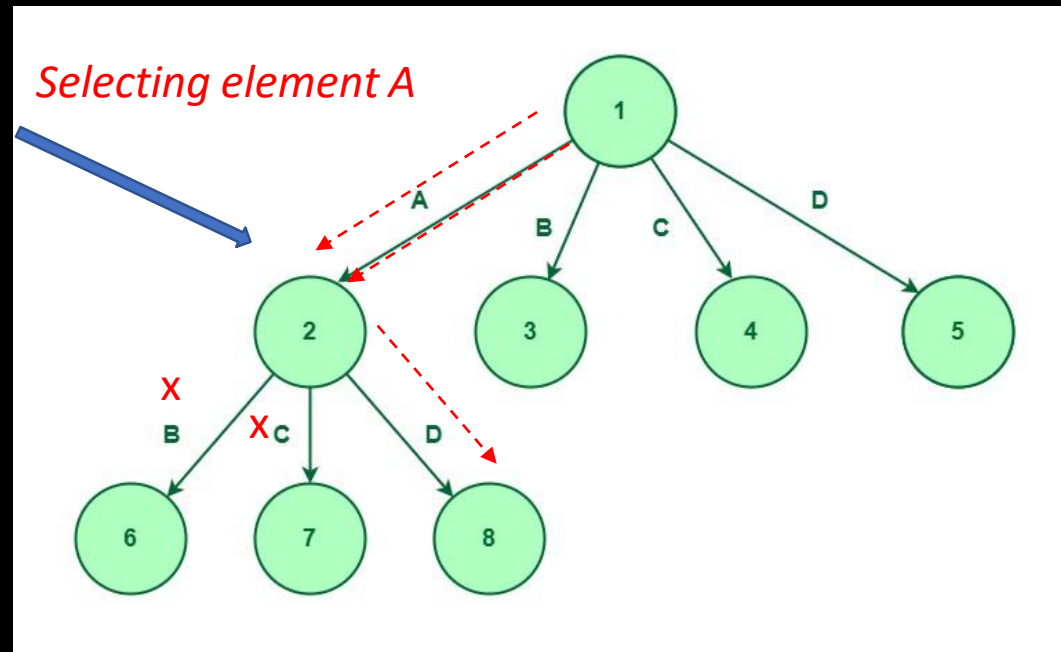
State Space tree for set {A, B, C, D}

- The above diagram shows that we first consider element A, then element B, then element C and finally we will consider the last element which is D. We are performing BFS while exploring the nodes.

Branch Bound: FIFO

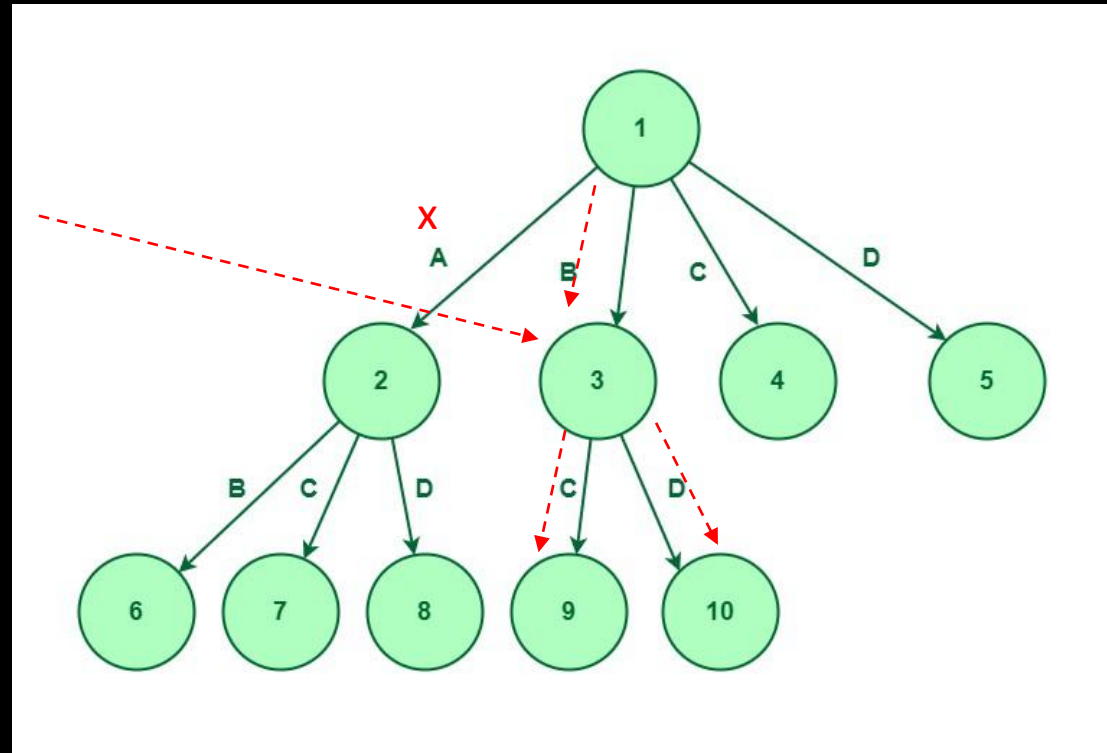
- once the **first level is completed**. We consider the **first element**, then we can consider **either B, C, or D**.
 - example
 - If we **select** the elements **A and D only**, then it says that we are selecting elements A and D and we are **NOT** considering elements **B and C**

FIFO start point



Branch Bound : FIFO

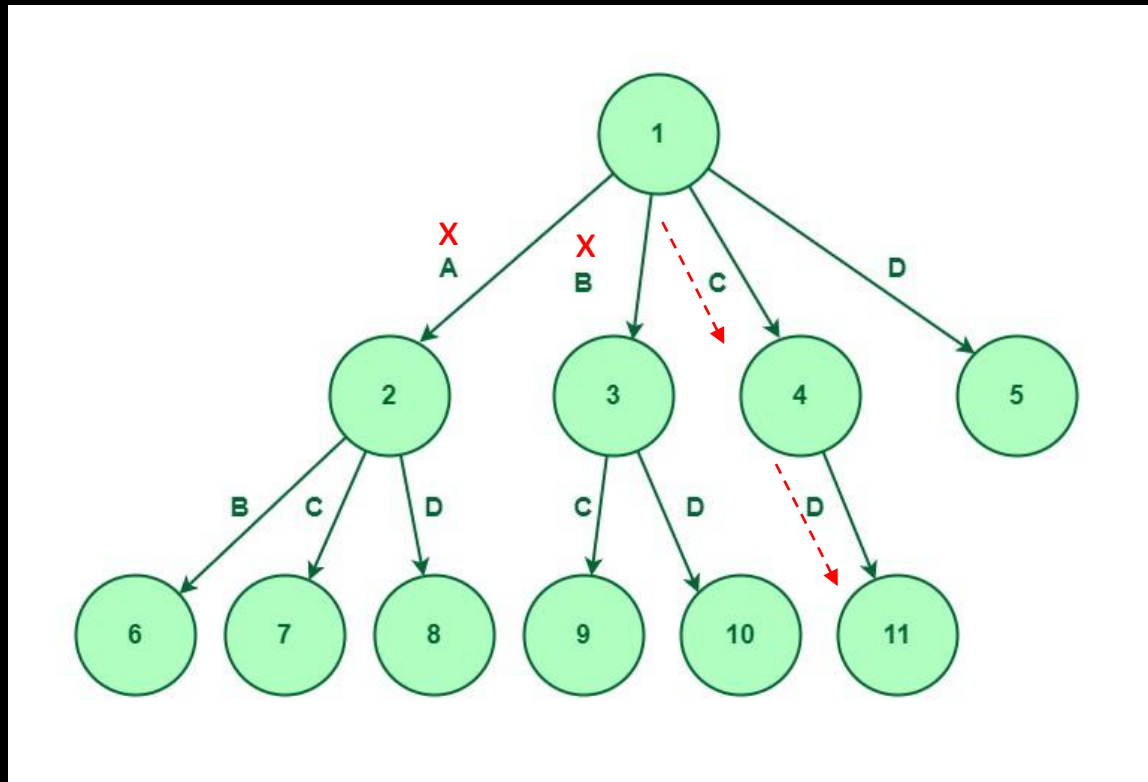
- Now, we will expand **node 3**, as we have considered element **B and NOT** considered element **A**, so, we have **two options** to explore that is elements **C and D**. Let us create **nodes 9 and 10** for elements **C and D**, respectively.



*Next in the queue is 3
Considered element B and not
considered element A*

Branch Bound : FIFO

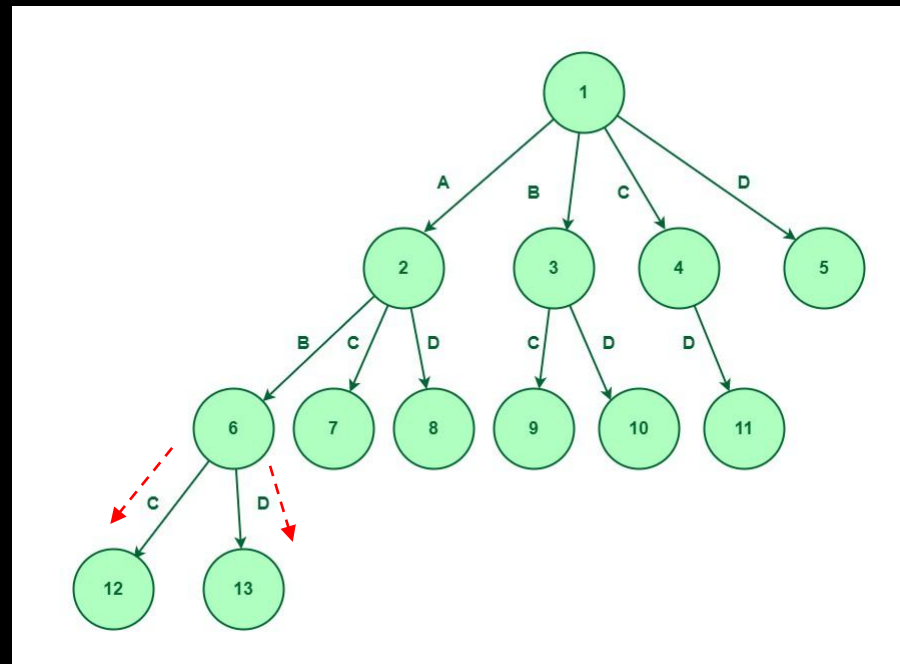
- Now, we will expand **node 4** as we have only **considered** elements **C** and **NOT** considered elements **A and B**, so, we have only one option to explore which is element **D**. Let us create node **11 for D**.



Considered elements C and not considered elements A and B

Branch Bound : FIFO

- Till **node 5**, we have **only** considered elements **D**, and **NOT** selected elements **A, B, and C**. So, We have **no more** elements to explore. Therefore on node 5, there **won't be any expansion**.
- Now, we will **expand node 6** as we have **considered** elements **A and B**, so, we have **only two options** to explore that is element **C and D**. Let us create **node 12 and 13** for **C and D** respectively.

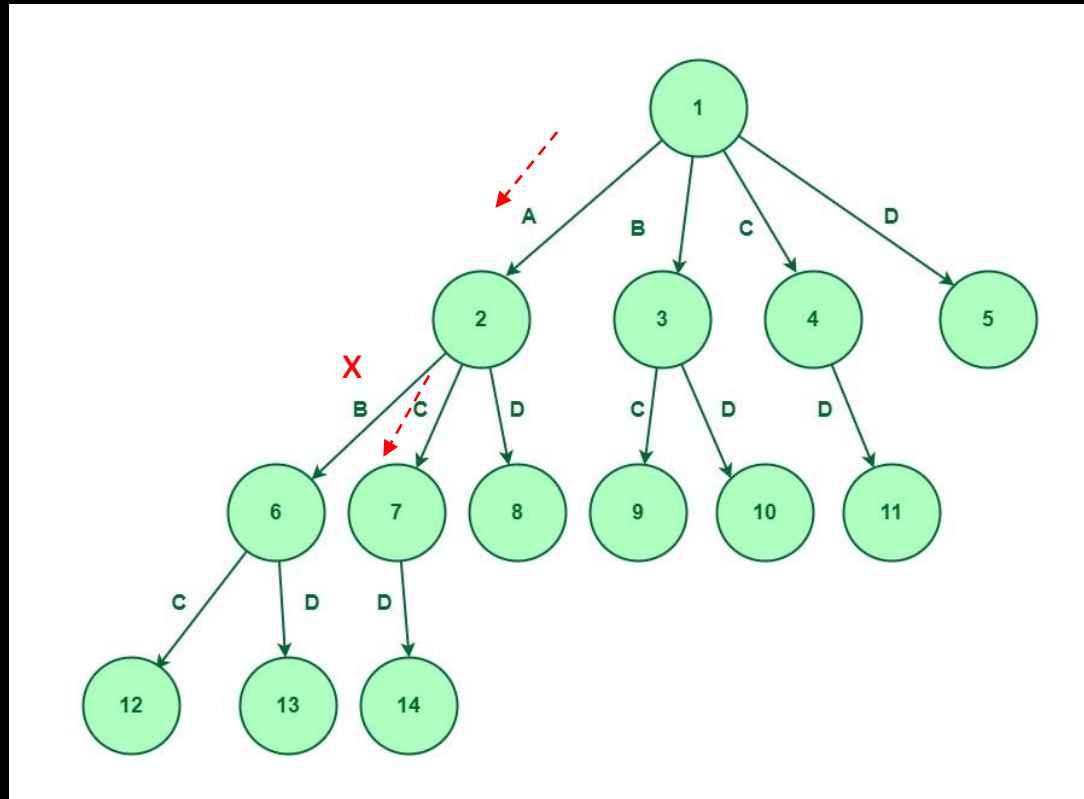


Next in the queue is 6

Expand node 6

Branch Bound : FIFO

- Now, we will expand **node 7** as we have **considered elements A and C** and **NOT** consider element **B**, so, we have only **one option** to explore which is **element D**. Let's create **node 14** for **D**.

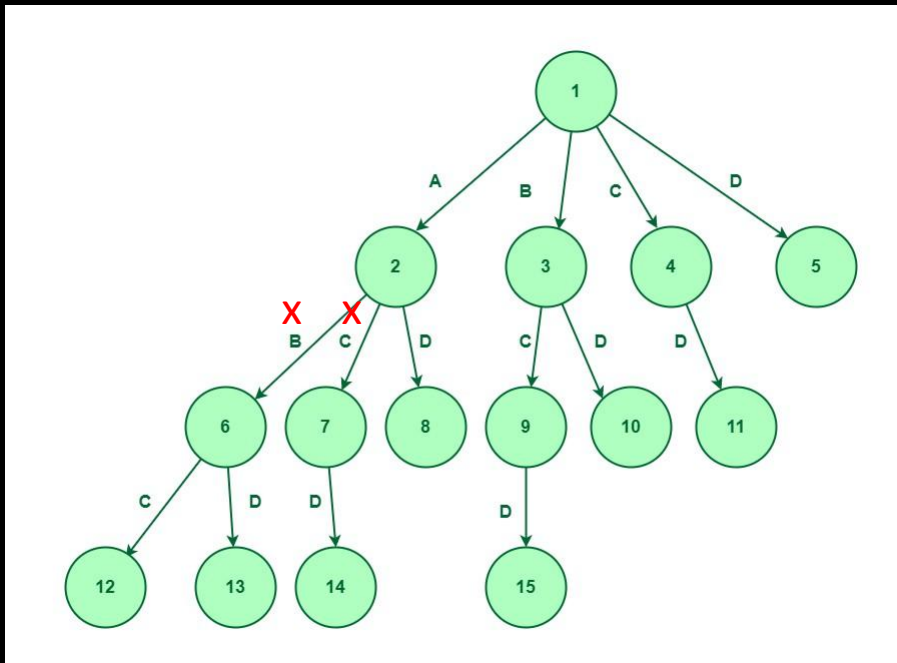


Next in the queue is 7

Expand node 7

Branch Bound : FIFO

- Till **node 8**, we have **considered** elements **A and D**, and **NOT** selected elements **B and C**, So, We have **no more elements to explore**, Therefore on node 8, there **won't be any expansion**.
- Now, we will expand **node 9** as we have considered elements B and C and **not considered element A**, so, we have only one option to explore which is **element D**. Let's create node **15 for D**.

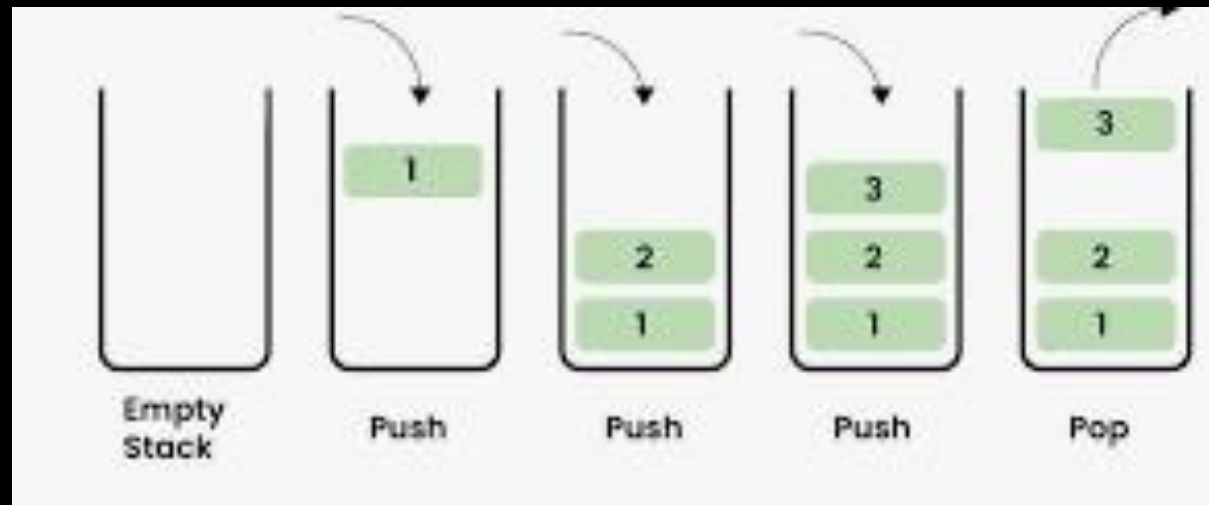


← state space tree

Expand node 9

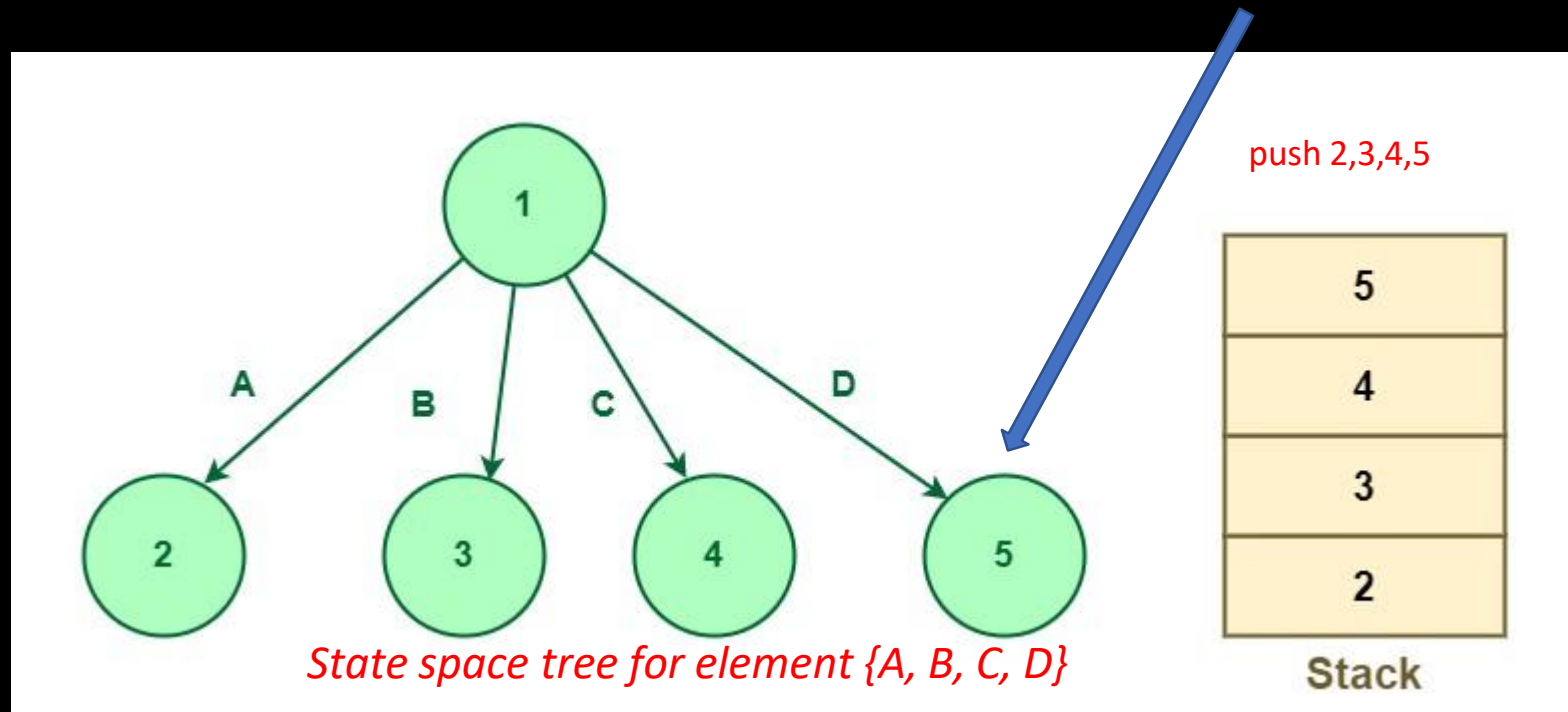
Branch Bound: LIFO

- 2. LIFO Branch and Bound
- The Last-In-First-Out approach for this problem uses stack in creating the **state space tree**.
 - When **nodes are added to a state space tree, they are added to a stack.**
 - **After all nodes of a level have been added, we pop the topmost element** from the stack and explore it.
- LIFO Branch and Bound uses a **Stack** to **explore next node** in BFS fashion



Branch Bound: LIFO

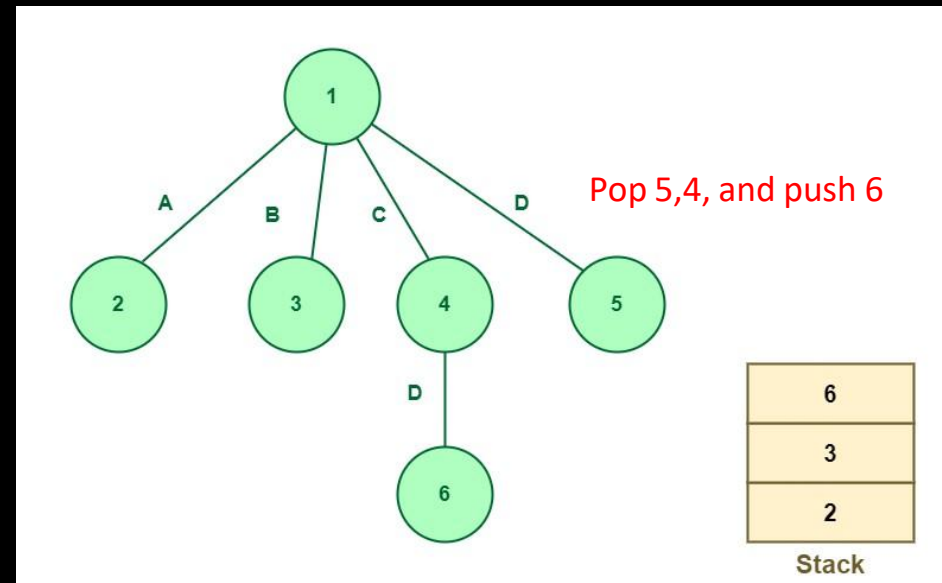
- For a given set {A, B, C, D}, the state space tree will be constructed as follows :



- first consider element A, then element B, then element C and finally we will consider the last element which is D

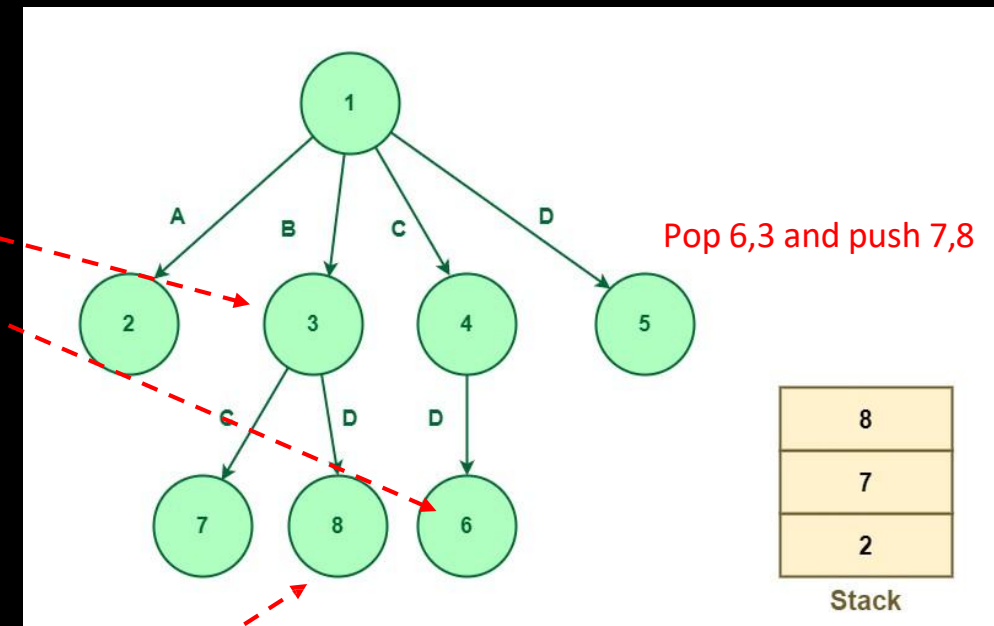
Branch Bound: LIFO

- Now the **expansion** would be based on the node that appears **on the top of the stack**. Since **node 5** appears on the **top of the stack**, so we will **expand node 5**. We will **pop out node 5** from the stack. Since **node 5** is in the last element, i.e., D so there is **no further scope for expansion**.
- The next node that appears on the **top of the stack is node 4**. **Pop-out node 4 and expand**. On expansion, element D will be considered and **node 6** will be **added** to the stack shown below:



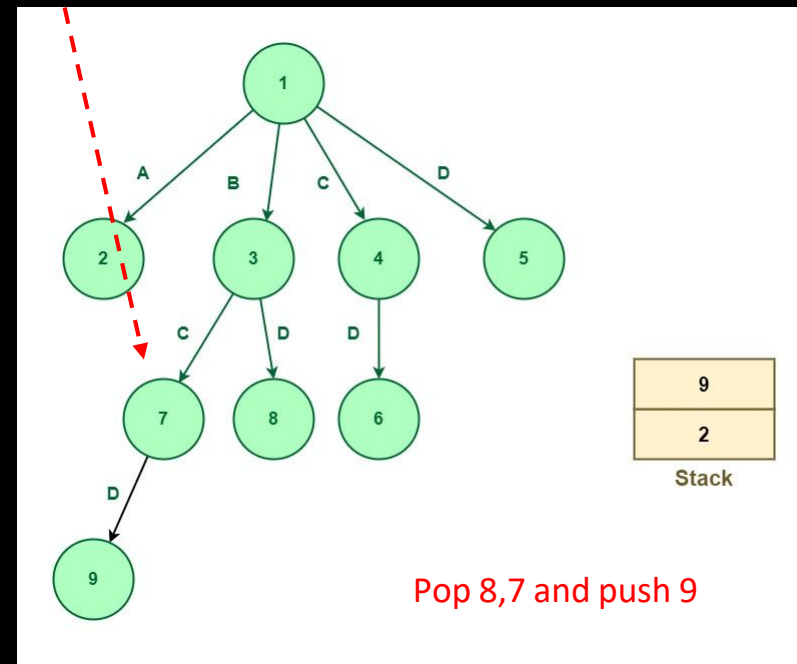
Branch Bound: LIFO

- The **next node is 6** which is to be expanded. Pop-out node 6 and expand. Since node 6 is in the **last element**, i.e., **D** so there is no further scope for expansion.
- The next node to be expanded is **node 3**. Since **node 3** works on element B so **node 3** will be expanded to **two nodes**, i.e., 7 and 8 working on **elements C and D**, respectively. **Nodes 7 and 8** will be **pushed** into the stack.
- The next node that appears on the top of the stack is node 8. **Pop-out node 8** and expand. Since node 8 works on element **D** so there is no further scope for the expansion.



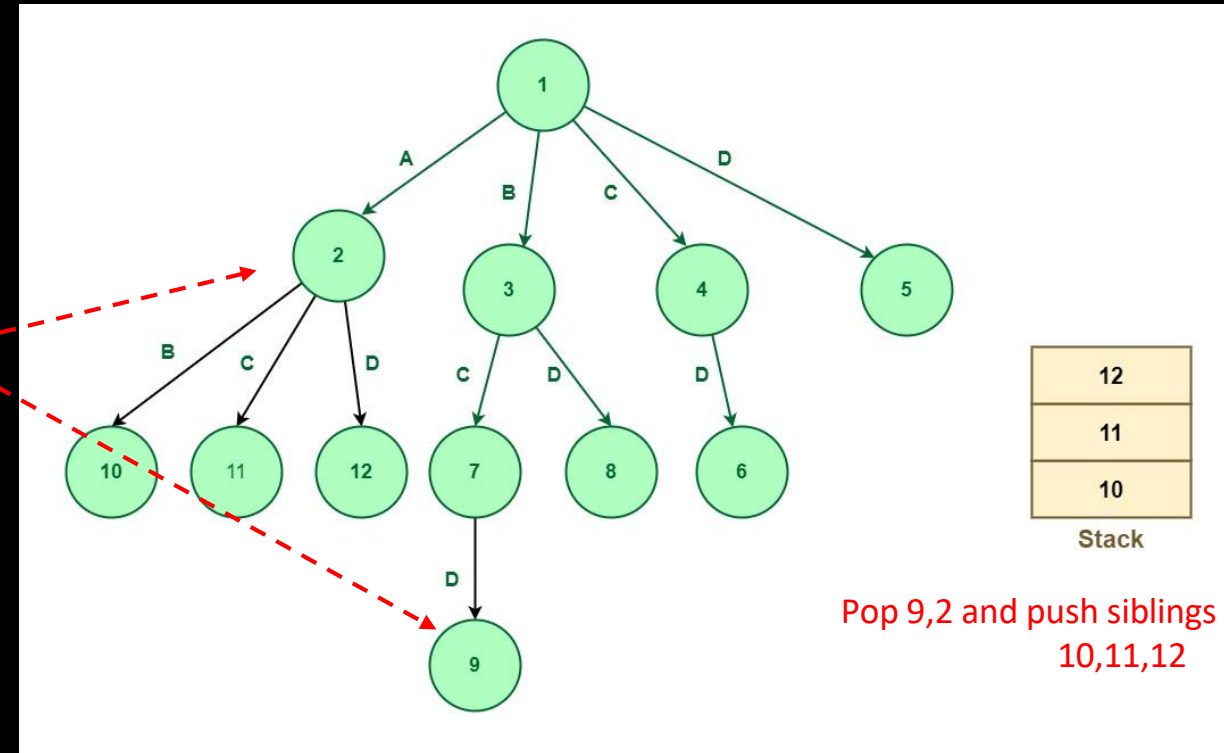
Branch Bound: LIFO

- The next node that appears on the top of the stack is **node 7**. **Pop-out node 7 and expand**. Since node 7 works on element C so node 7 will be further expanded to **node 9** which works on element D and **node 9 will be pushed into the stack**.
- The **next node is 6** which is to be expanded. Pop-out node 6 and expand. Since node 6 is in the last element, i.e., D so there is no further scope for expansion.



Branch Bound: LIFO

- The **next node** that appears on the top of the stack is **node 9**. Since node 9 works on element **D**, there is **no** further scope for **expansion**.
- The next node that appears on the top of the stack is **node 2**. Since node 2 works on the **element A** so it means that node 2 can be further expanded. It can be expanded up to three nodes named **10, 11, 12** working on elements **B, C, and D** respectively. There new nodes will be pushed into the stack shown as below:



Expand node 2

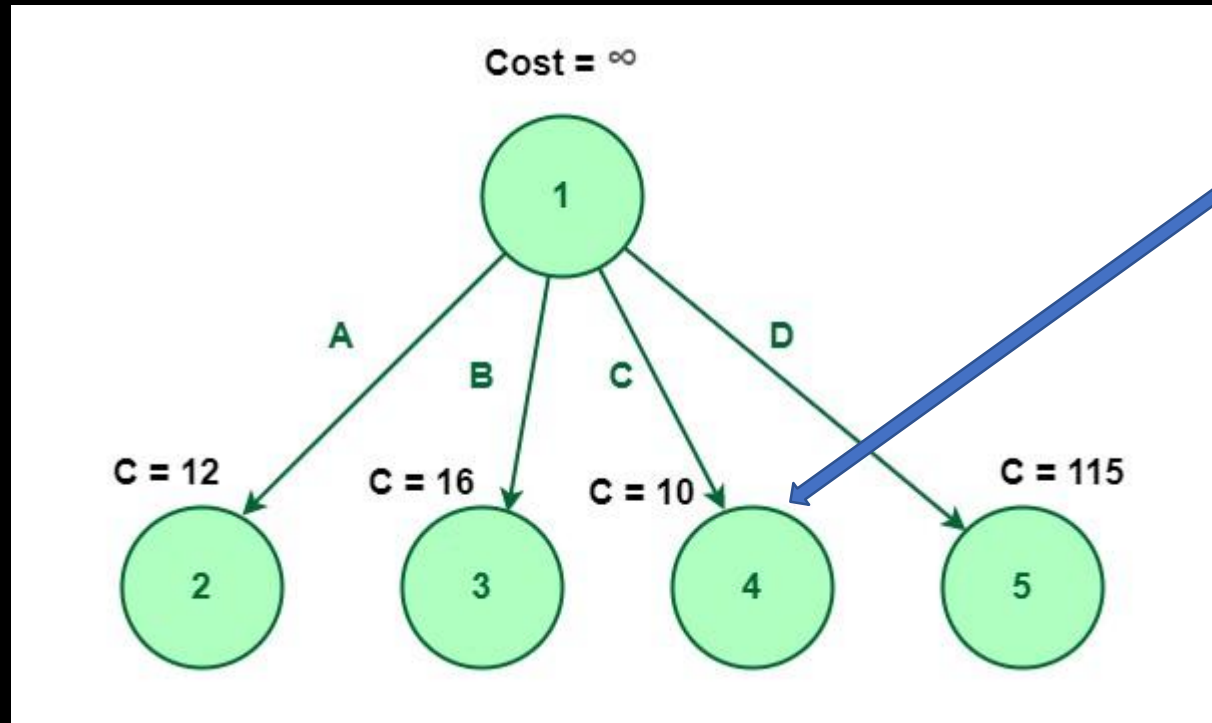
In the above method, we explored all the nodes using the stack that follows the LIFO principle.

Branch Bound: Least Cost-Branch and Bound

- Uses a cost (bound) function to **guide exploration of the state-space tree**
- **FIFO & LIFO also compute bound**, but **only for pruning** but **NOT for choosing the next node**
 - FIFO uses queue (BFS-style)
 - LIFO uses stack (DFS-style)
- LC Search **always selects the most promising live node** (least cost) to explore next (i.e. best-first search)
- Pruning Rule (minimization problems):
 - If a node's **bound $\geq$ cost of best solution** found so far **then prune** (node & its children can't improve the answer)
- Pruning **avoids wasted exploration**, hence more **efficient** search
- Generally more efficient than FIFO/LIFO. However NOT guaranteed faster in every case

Branch Bound: Least Cost-Branch and Bound (LCBB)

- Let us first consider **node 1** having **cost infinity**, shown below:
- In the following diagram, **node 1** is **expanded** into four nodes named **2, 3, 4, and 5**.

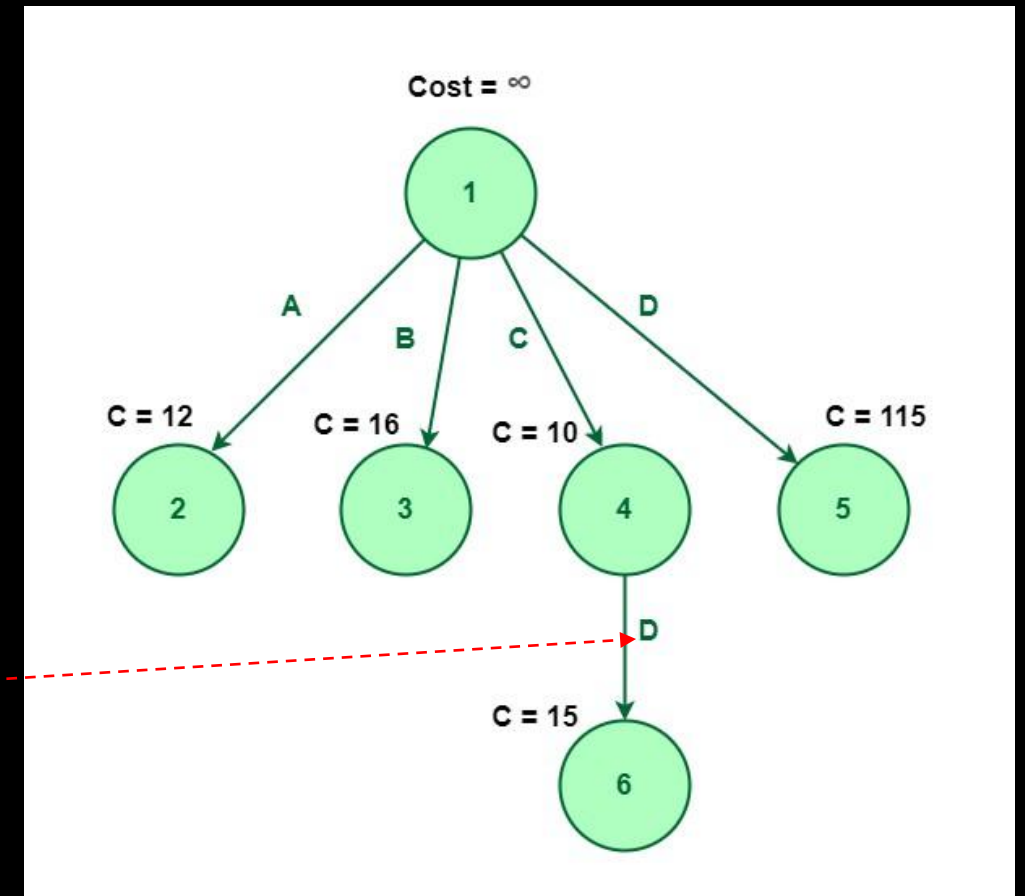


LC start point

- Usually in LCBB implementations, before computing the actual bound (reduced cost, lower bound, etc.), cost is **temporarily initialized** to ∞ .
- Then, if a valid cost is found, it is updated. If not, it remains ∞
- Nodes with ∞ are **effectively excluded** from consideration.

Branch Bound: Least Cost-Branch and Bound

- Assume that cost of the nodes 2, 3, 4, and 5 are **12, 16, 10, and 115** respectively.
In this method, we will **explore** the node which is having the **least cost**.
- In the above figure, we can observe that the node with a **minimum cost is node 4**. So, we will explore node 4 having a cost of 10.
- During exploring **node 4 which is element C**, we can notice that there is **only one possible element** that remains **unexplored** which is **D** (i.e, we **already decided not to select elements A, and B**).
- So, it will get expanded to one single element D, let's say this node number is 6.

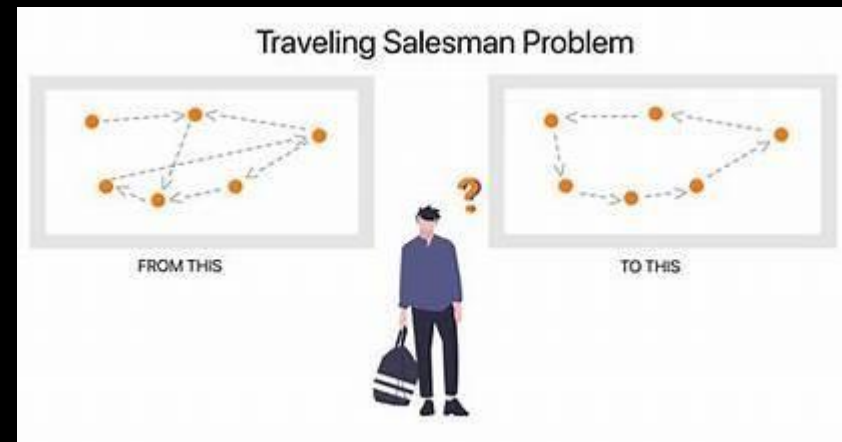


Exploring node 4 which is element C

Now, Node 6 has no element left to explore. So, there is no further scope for expansion.
Hence the element {C, D} is the optimal way to choose for the least cost.

Branch Bound Example TSP

- Traveling Salesman Problem - Branch and Bound
- https://www.youtube.com/watch?v=1FEP_sNb62k
- <https://www.baeldung.com/cs/branch-and-bound>



Traveling Salesman

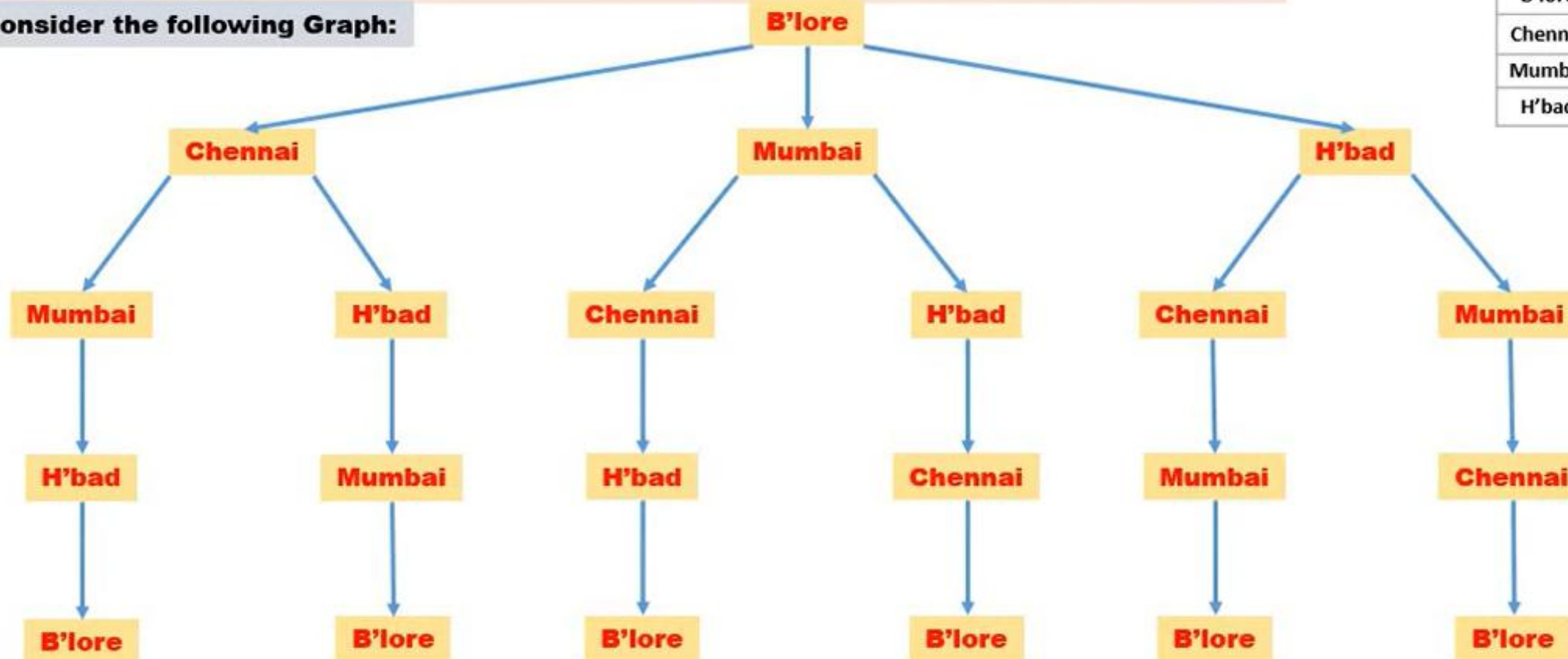
Traveling Salesman

The Traveling Salesman should start from a source city, travel through all the other cities only once and at last travel to the source city

Now, the Traveling Salesman should travel in such a manner that the total traveling cost or distance is minimum

Consider the following Graph:

	B'lore	Chennai	Mumbai	H'bad
B'lore	0	350	1000	600
Chennai	350	0	1400	700
Mumbai	1000	1400	0	700
H'bad	600	700	700	0



<https://www.youtube.com/watch?v=HjSbaKF8Gi0>

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

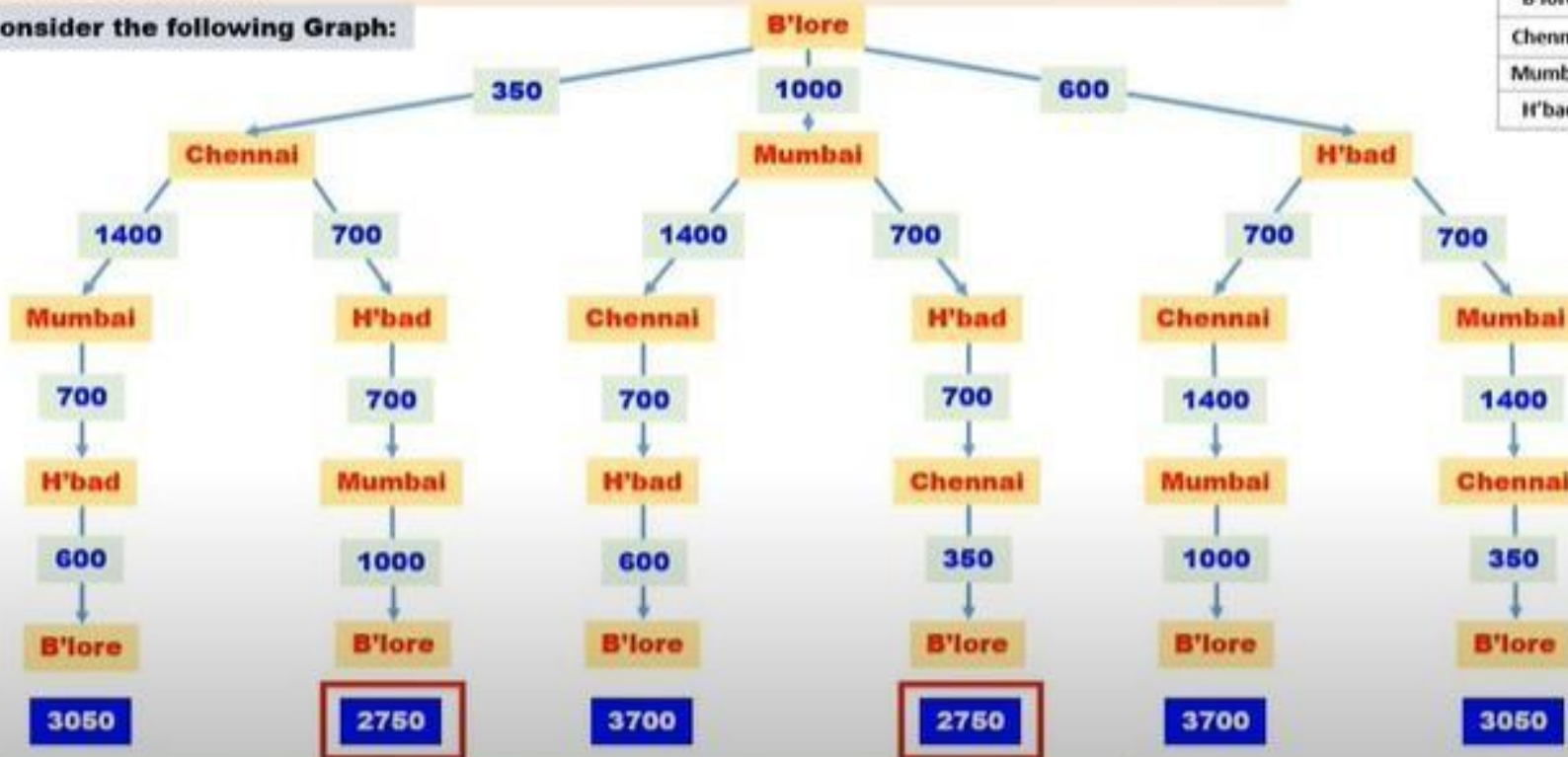
In Traveling Salesman Problem, there are n cities and a Traveling Salesman

The Traveling Salesman should start from a source city, travel through all the other cities only once and at last travel to the source city

Now, the Traveling Salesman should travel in such a manner that the total traveling cost or distance is minimum

Consider the following Graph:

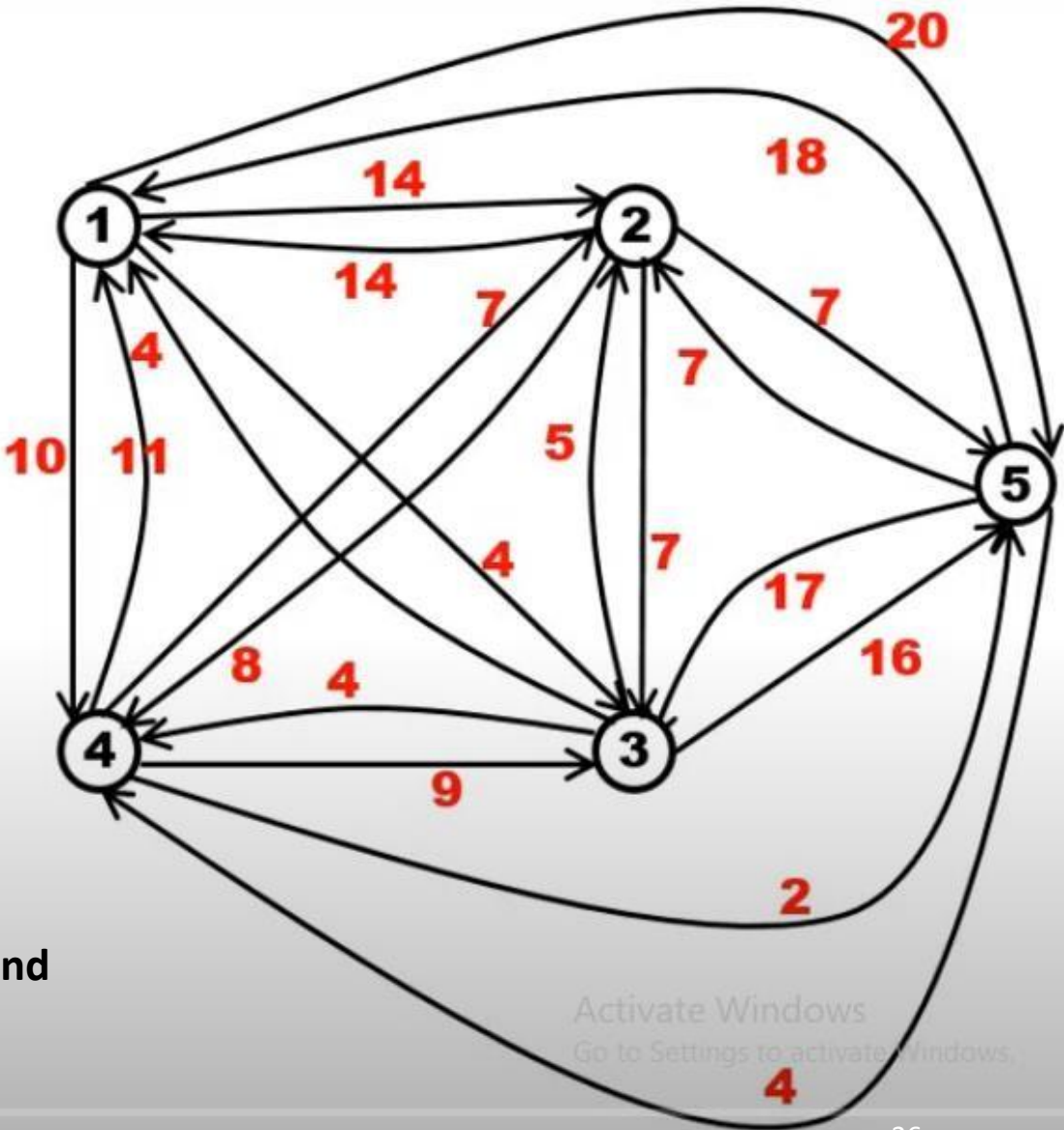
	B'lore	Chennai	Mumbai	H'bad
B'lore	—	350	1000	600
Chennai	350	—	1400	700
Mumbai	1000	1400	—	700
H'bad	600	700	700	—



The Cost Matrix for the Graph is as follows:

	1	2	3	4	5
1	∞	14	4	10	20
2	14	∞	7	8	7
3	4	5	∞	4	16
4	11	7	9	∞	2
5	18	7	17	4	∞

Consider the following Graph:



Branch Bound: Least Cost-Branch and Bound

Branch Bound: Least Cost-Branch and Bound

The First Step is to get a Reduced Matrix by Reducing the Cost Matrix

To Reduce the Cost Matrix, We have to do Row Reduction and then Column Reduction

To do Row Reduction, find the minimum number in each Row and this number is subtracted from all the numbers in that Row

	1	2	3	4	5
1	∞	10	0	6	16
2	7	∞	0	1	0
3	0	1	∞	0	12
4	9	5	7	∞	0
5	14	3	13	0	∞

4

7

4

2

4

Row Reduction Cost =

$$4 + 7 + 4 + 2 + 4 = 21$$

To do Column Reduction, find the minimum number in each Column and this number is subtracted from all the numbers in that Column

	1	2	3	4	5
1	∞	9	0	6	16
2	7	∞	0	1	0
3	0	0	∞	0	12
4	9	4	7	∞	0
5	14	2	13	0	∞

0 1 0 0 0

$$\text{Column Reduction Cost} = 0 + 1 + 0 + 0 + 0 = 1$$

Cost Matrix

	1	2	3	4	5
1	∞	14	4	10	20
2	14	∞	7	8	7
3	4	5	∞	4	16
4	11	7	9	∞	2
5	18	7	17	4	∞

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

The First Step is to get a Reduced Matrix by Reducing the Cost Matrix

To Reduce the Cost Matrix, We have to do Row Reduction and then Column Reduction

To do Row Reduction, find the minimum number in each row and this number is subtracted from all the numbers in that row

	1	2	3	4	5	
1	∞	10	0	6	16	4
2	7	∞	0	1	0	7
3	0	1	∞	0	12	4
4	9	5	7	∞	0	2
5	14	3	13	0	∞	4

Row Reduction Cost =
 $4 + 7 + 4 + 2 + 4 = 21$

To do Column Reduction, find the minimum number in each column and this number is subtracted from all the numbers in that column

	1	2	3	4	5	
1	∞	9	0	6	16	
2	7	∞	0	1	0	
3	0	0	∞	0	12	
4	9	4	7	∞	0	
5	14	2	13	0	∞	
	0	1	0	0	0	

Column Reduction Cost = $0 + 1 + 0 + 0 + 0 = 1$

Cost Matrix

	1	2	3	4	5
1	∞	14	4	10	20
2	14	∞	7	8	7
3	4	5	∞	4	16
4	11	7	9	∞	2
5	18	7	13	0	∞

Reduced Cost = Row
 + Column

= $21 + 1 = 22$

This is the
Reduced
Cost

This is the
Reduced
Matrix

General Algorithm

Let us Consider the Reduced Matrix

We have the Cost Matrix value 22 for this Reduced Matrix

We Compute the Reduced Cost for each Node [corresponding to an edge (i, j)] in the State Space Tree as follows:

- 1: Change all entries in Row i and Column j to ∞ to avoid any edges leaving i or entering j**
- 2. Set (j,1) to ∞ to prevent the use of edge (j, 1)**
- 3. Then Reduce all rows and columns in the resulting Matrix except for the Rows and Columns containing only ∞**
- 4. The Reduced Cost for the Node corresponding to edge (i, j) = Reduced Cost at Node j + Cost of [i, j] + Reduced Cost at Node i**

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

Let us Consider the Reduced Matrix

We have the Cost Matrix value 22 for this Reduced Matrix

We Compute the Reduced Cost for each Node [corresponding to an edge (i, j)] in the State Space Tree as follows:

1: Change all entries in Row i and Column j to ∞ to avoid any edges leaving i or entering j

2. Set (i,1) to ∞ to prevent the use of edge (j, 1)

Row 1 and Column 2 entries

should be made ∞

Also, (2,1) should be made ∞

columns in the resulting Matrix except for the Rows and

Node corresponding to edge (i, j) = Reduced Cost at Node j
Cost at Node i

	1	2	3	4	5
1	∞	9	0	6	16
2	7	∞	0	1	0
3	0	0	∞	0	12
4	9	4	7	∞	0
5	14	2	13	0	∞

Node 1: Reduced Cost is 22

Node 12: For edge (1, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2		∞			
3		∞	∞		
4		∞		∞	
5		∞			∞

← Step 1

Node 12: For edge (1, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞			
3		∞	∞		
4		∞		∞	
5		∞			∞

← Step 2

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

Let us Consider the Reduced Matrix

We have the Cost Matrix value 22 for this Reduced Matrix

We Compute the Reduced Cost for each Node [corresponding to an edge (i, j)] in the State Space Tree as follows:

- 1: Change all entries in Row i and Column j to ∞ to avoid any edges leaving i or entering j
2. Set (j,1) to ∞ to prevent the use of edge (j, 1)
3. Then Reduce all rows and columns in the resulting Matrix except for the Rows and Columns containing only ∞
4. The Node corresponding to edge (i, j) = Reduced Cost at Node j + Reduced Cost at Node i

Now, We shall Reduce this Matrix

Node 1: Reduced Cost is 22

	1	2	3	4	5
1	∞	9	0	6	16
2	7	∞	0	1	0
3	0	0	∞	0	12
4	9	4	7	∞	0
5	14	2	13	0	∞

Node 1: Reduced Cost is 22

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	1	0
3	0	∞	∞	0	12
4	9	∞	7	∞	0
5	14	∞	13	0	∞

Reduced Cost at Node 12 (1, 2)

=

Copy remaining values are copied from reduced matrix after inserting inf

Branch Bound: Least Cost-Branch and Bound

Node 12: For edge (1, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	1	0
3	0	∞	∞	0	12
4	9	∞	7	∞	0
5	14	∞	13	0	∞

Reduced Cost at Node 12 (1, 2)

= 0

Total Reduced cost at Node 12(Row 1, column) = 0 + 0

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

Let us Consider the Reduced Matrix

We have the Cost Matrix value 22 for this Reduced Matrix

We Compute the Reduced Cost for each Node [corresponding to an edge (i, j)] in the State Space Tree as follows:

- 1: Change all entries in Row i and Column j to ∞ to avoid any edges leaving i or entering j
2. Set (j,1) to ∞ to prevent the use of edge (j, 1)
3. Then Reduce all rows and columns in the resulting Matrix except for the Rows and Columns containing only ∞
4. The Reduced Cost for the Node corresponding to edge (i, j) = Reduced Cost at Node j + Cost of [i, j] + Reduced Cost at Node i

	1	2	3	4	5
1	∞	9	0	6	16
2	7	∞	0	1	0
3	0	0	∞	0	12
4	9	4	7	∞	0
5	14	2	13	0	∞

Node 1: Reduced Cost is 22

Node 12: For edge (1, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	1	0
3	0	∞	∞	0	12
4	9	∞	7	∞	0
5	14	∞	13	0	∞

Reduced Cost at Node 12 (1, 2)

$$= 0 + 9 + 22 = 31$$

4. The Reduced Cost for the Node corresponding to edge (i, j) = Reduced Cost at Node j + Cost of [i, j] + Reduced Cost at Node i

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

Let us Consider the Reduced Matrix

We have the Cost Matrix value 22 for this Reduced Matrix

We Compute the Reduced Cost for each Node [corresponding to an edge (i, j)] in the State Space Tree as follows:

- 1: Change all entries in Row i and Column j to ∞ to avoid any edges leaving i or entering j
2. Set (j,1) to ∞ to prevent the use of edge (j, 1)
3. Then Reduce all rows and columns in the resulting Matrix except for the Rows and Columns containing only ∞
4. The Reduced Cost for the Node corresponding to edge (i, j) = Reduced Cost at Node j + Cost of [i, j] + Reduced Cost at Node i

	1	2	3	4	5
1	∞	9	0	6	16
2	7	∞	0	1	0
3	0	0	∞	0	12
4	9	4	7	∞	0
5	14	2	13	0	∞

Now, We shall Reduce this Matrix
Node 15: For edge (1, 5)

Node 12: For edge (1, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	1	0
3	0	∞	∞	0	12
4	9	∞	7	∞	0
5	14	∞	13	0	∞

Reduced Cost at Node 12 (1, 2)
= $0 + 9 + 22 = 31$

Node 13: For edge (1, 3)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

Reduced Cost at Node 13 (1, 3)
= $7 + 0 + 22 = 29$

Node 14: For edge (1, 4)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	0
3	0	0	∞	∞	12
4	∞	4	7	∞	0
5	12	0	11	∞	∞

Reduced Cost at Node 14 (1,4)
= $2 + 6 + 22 = 30$

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	1	∞
3	0	0	∞	0	∞
4	5	0	3	∞	∞
5	∞	2	13	0	∞

Reduced Cost at Node 15 (1,5)
= $4 + 16 + 22 = 42$

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

Let us Consider the Reduced Matrix

Now can consider 1->2, 1->3, 1->4 and 1->5

We have the Cost Matrix value 22 for this Reduced Matrix

We Compute the Reduced Cost for each Node [corresponding to an edge (i, j)] in the State Space Tree as follows:

- 1: Change all entries in Row i and Column j to ∞ to avoid any edges leaving i or entering j
2. Set (j,1) to ∞ to prevent the use of edge (j, 1)
3. Then Reduce all rows and columns in the resulting Matrix except for the Rows and

	1	2	3	4	5
1	∞	9	0	6	16
2	7	∞	0	1	0
3	0	0	∞	0	12
4	9	4	7	∞	0

This is the Reduced Matrix for Node 12 [For edge (1, 2)]

This is the Reduced Matrix for Node 13 [For edge (1, 3)]

This is the Reduced Matrix for Node 14 [For edge (1, 4)]

This is the Reduced Matrix for Node 15 [For edge (1, 5)]

Node 12: For edge (1, 2)

Node 13: For edge (1, 3)

Node 14: For edge (1, 4)

Node 15: For edge (1, 5)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	1	0
3	0	∞	∞	0	12
4	9	∞	7	∞	0
5	14	∞	13	0	∞

Reduced Cost at Node 12 (1, 2)
= 0 + 9 + 22 = 31

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

Reduced Cost at Node 13 (1, 3)
= 7 + 0 + 22 = 29

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	0
3	0	0	∞	∞	12
4	∞	4	7	∞	0
5	12	0	11	∞	∞

Reduced Cost at Node 14 (1, 4)
= 2 + 6 + 22 = 30

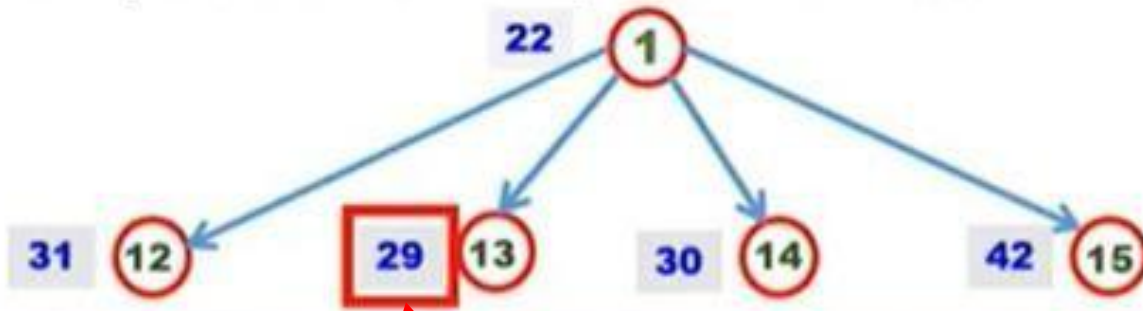
	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	1	∞
3	0	0	∞	0	∞
4	5	0	3	∞	∞
5	∞	2	13	0	∞

Reduced Cost at Node 15 (1, 5)
= 4 + 16 + 22 = 42

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

Now, We have the Partial Space Tree as follows:



In Least Cost Branch and Bound, we have to explore and branch the Node with Least Cost among all the Leaf Nodes

Now, the Least Cost among 31, 29, 30 and 42 is 29. Therefore, we have to explore the Node 13 [edge (1, 3)]

Reduced Matrix
for Node 13 (1,3) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

Reduced Cost = 29

Partial SST

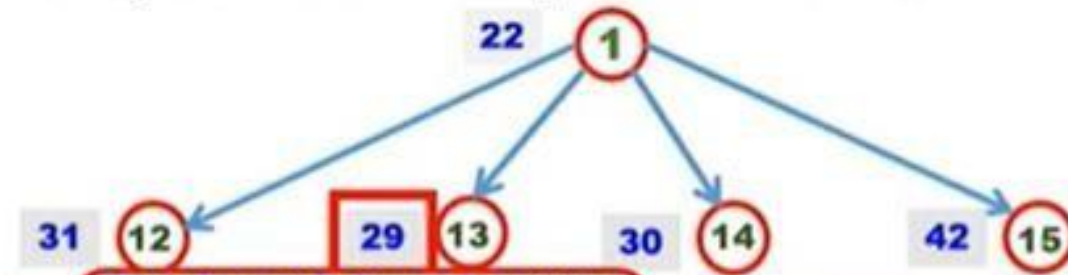
**Note the node naming convention. 12: 1->2; 13: 1->3 etc

Least Cost node 13 having a cost of 29

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

Now, We have the Partial Space Tree as follows:



Reduced Matrix for Node 13 (1,3) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

Reduced Cost = 29

In L...
Lea...
Now...
the Node...
age (1, 3)]

Row 3 and Column 2 entries should be made ∞
Also, (2,1) should be made ∞

We have to explore and branch the Node with 30 and 42 is 29. Therefore, we have to explore

Node 132: For edge (3, 2)

	1	2	3	4	5
1	∞	∞			
2	∞	∞			
3	∞	∞	∞	∞	∞
4		∞		∞	
5		∞			∞

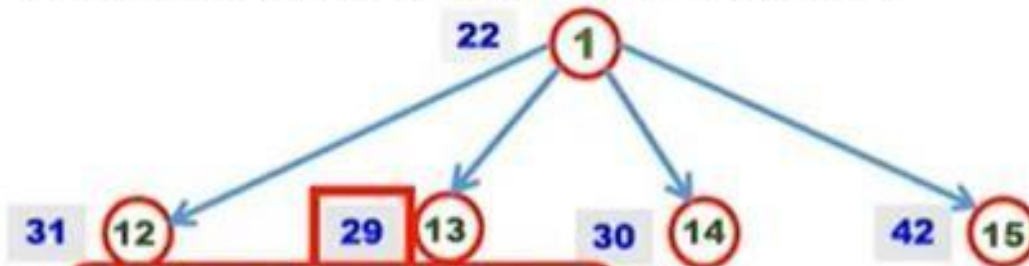
Note: matrix for Node 132 and Edge (3,2). i.e Travel 1->3->2

Can not go from 3 to 1, hence explore 3,2

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

Now, We have the Partial Space Tree as follows:



Reduced Matrix
for Node 13 (1,3) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

Reduced Cost = 29

Row 3 and Column 2 entries
should be made ∞
Also, (2,1) should be made ∞

We have to explore and branch the Node with

30 and 42 is 29. Therefore, we have to explore

Node 132: For edge (3, 2)

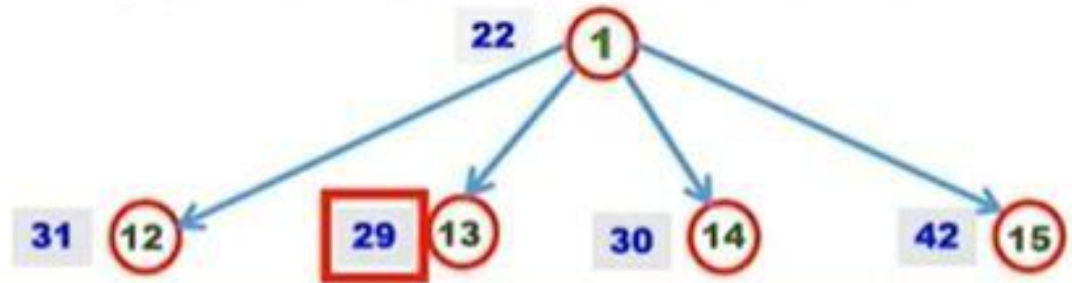
	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	∞	1	0
3	∞	∞	∞	∞	∞
4	2	∞	∞	∞	0
5	7	∞	∞	0	∞

Copy remaining values are copied from reduced matrix after inserting inf

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

Now, We have the Partial Space Tree as follows:



Reduced Matrix
for Node 13 (1,3) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

Reduced Cost = 29

In Least Cost Branch and Bound, we have to explore and branch the Node with Least Cost.

Now, the cost of nodes 29, 30 and 42 is 29. Therefore, we have to explore the Node with cost 29.

Now, We shall
Reduce this Matrix

Node 132: For edge (3, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	∞	1	0
3	∞	∞	∞	∞	∞
4	2	∞	∞	∞	0
5	7	∞	∞	0	∞

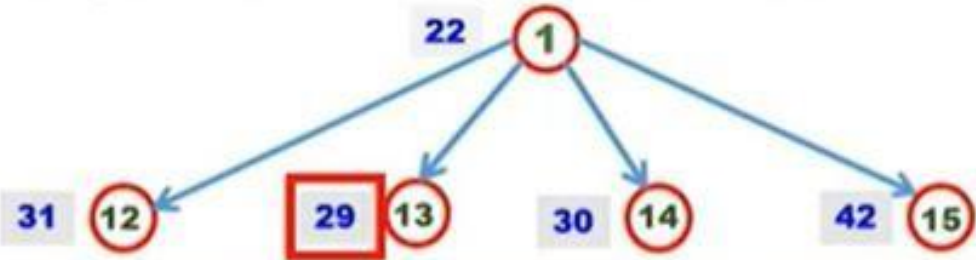
Node 132: For edge (3, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	∞	1	0
3	∞	∞	∞	∞	∞
4	2	∞	∞	∞	0
5	7	∞	∞	0	∞

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

Now, We have the Partial Space Tree as follows: Now can consider 3->2, 3->4 and 3->5



Reduced Matrix
for Node 13 (1,3) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

Reduced Cost = 29

In Least Cost Branch and Bound, we have to explore and branch the Node with Least Cost among all the Leaf Nodes

Now, the Least Cost among 31, 29, 30 and 42 is 29. Therefore, we have to explore the Node 13 [edge (1, 3)]

Node 132: For edge (3, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	∞	1	0
3	∞	∞	∞	∞	∞
4	0	∞	∞	∞	0
5	5	∞	∞	0	∞

Reduced Cost at Node 132 (3, 2)
= 2 + 0 + 29 = 31

Node 134: For edge (3, 4)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	∞	0
3	∞	∞	∞	∞	∞
4	∞	4	∞	∞	0
5	5	0	∞	∞	∞

Reduced Cost at Node 134 (3, 4)
= 2 + 0 + 29 = 31

Node 135: For edge (3, 5)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	∞
3	∞	∞	∞	∞	∞
4	0	0	∞	∞	∞
5	∞	0	∞	0	∞

Reduced Cost at Node 135 (3, 5)
= 4 + 12 + 29 = 45

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

Now, We have the Partial Space Tree as follows:

22 (1)

31 (12) 29 (13) 30 (14) 42 (15)

Now can consider 3->2, 3->4 and 3->5

Reduced Matrix for Node 13 (1,3) →

	1	2	3	4	5
1		∞	∞	∞	∞
2	0		∞	1	0
3	∞	0		0	12
4	2	4	∞		0
5	5	0	∞	0	

In L...
Lea...
Now the

This is the Reduced Matrix for Node 132 For edge (3, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞		∞	1	0
3	∞	∞		∞	∞
4	0	∞	∞		0
5	5	∞	∞	0	

Node 132: For edge (3, 2)

This is the Reduced Matrix for Node 134 For edge (3, 4)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0		∞	∞	0
3	∞	∞		∞	∞
4	∞	4	∞		0
5	5	0	∞	∞	

Node 134: For edge (3, 4)

This is the Reduced Matrix for Node 135 For edge (3, 5)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0		∞	1	∞
3	∞	∞		∞	∞
4	0	0	∞		∞
5	∞	0	∞	0	

Node 135: For edge (3, 5)

Reduced Cost at Node 132 (3, 2)
= 2 + 0 + 29 = 31

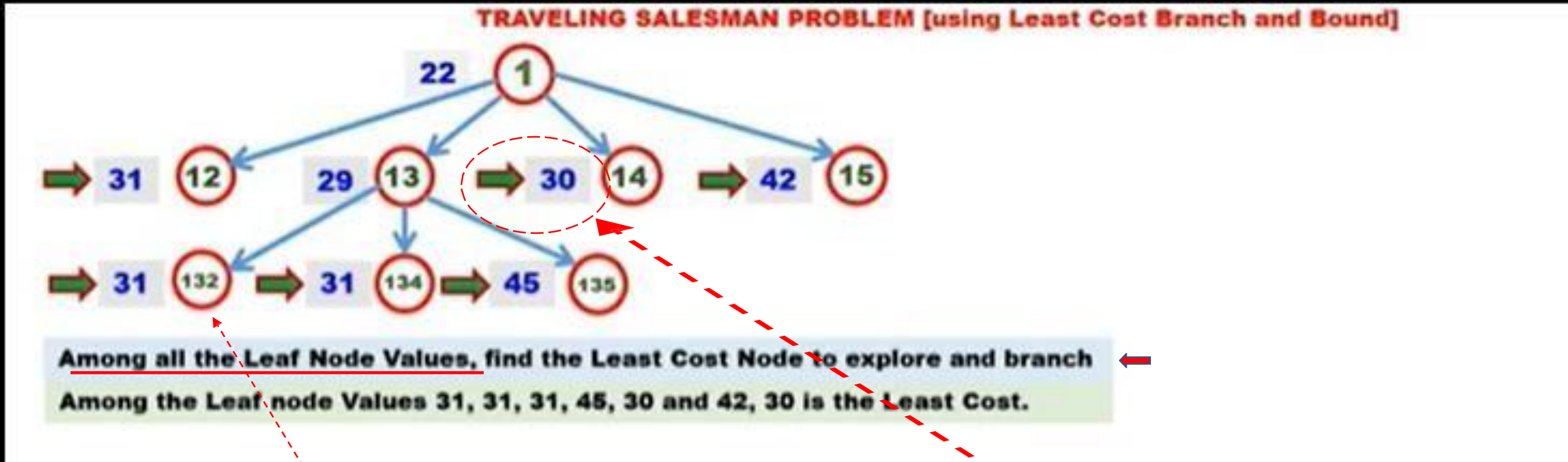
Reduced Cost at Node 134 (3, 4)
= 2 + 0 + 29 = 31

Reduced Cost at Node 135 (3, 5)
= 4 + 12 + 29 = 45

Cost = 29

Branch Bound: Least Cost-Branch and Bound

- State space tree for LC BnB

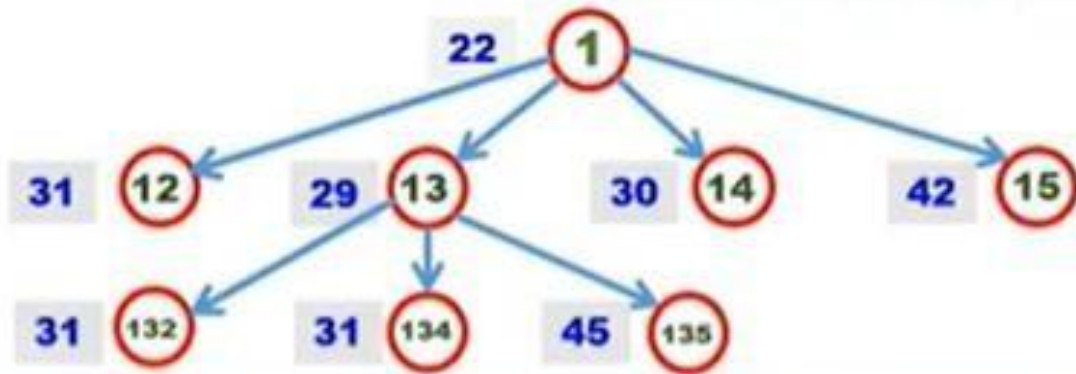


Note the naming convention 132 : 1->3->2

Select Least cost from leaves

Branch Bound: Least Cost-Branch and Bound

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



Reduced Matrix
for Node 14 (1, 4) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	0
3	0	0	∞	∞	12
4	∞	4	7	∞	0
5	12	0	11	∞	∞

Among all the Leaf Node Values, find the Least Cost Node to explore and branch
 Among the Leaf node Values 31, 31, 31, 45, 30 and 42, 30 is the Least Cost.
 Therefore, we explore and branch from Node 14

Node 12: For edge (1, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	1	0
3	0	∞	∞	0	12
4	9	∞	7	∞	0
5	14	∞	13	0	∞

Reduced Cost at Node 12 (1, 2)

= 0 + 9 + 22 = 31

Node 13: For edge (1, 3)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

Reduced Cost at Node 13 (1, 3)

= 7 + 0 + 22 = 29

Node 14: For edge (1, 4)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	0
3	0	0	∞	∞	12
4	∞	4	7	∞	0
5	12	0	11	∞	∞

Reduced Cost at Node 14 (1, 4)

= 2 + 6 + 22 = 30

Node 15: For edge (1, 5)

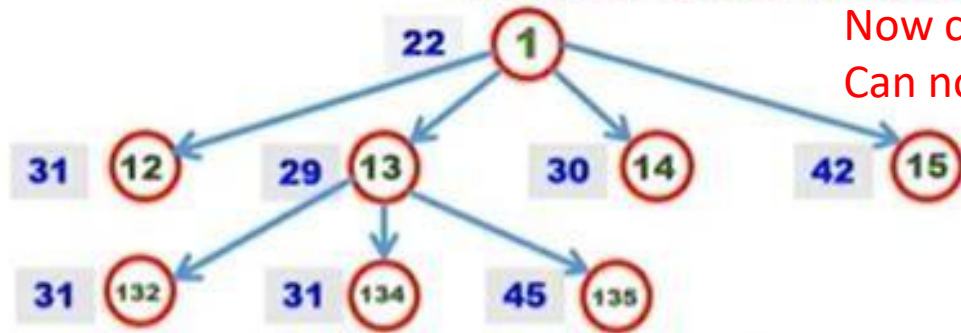
	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	1	∞
3	0	0	∞	0	∞
4	5	0	3	∞	∞
5	∞	2	13	0	∞

Reduced Cost at Node 15 (1, 5)

= 4 + 16 + 22 = 42

Take Reduced matrix for Node 14 from previous calculations

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



Now can consider 4->2, 4->3 and 4->5
Can not go back, hence no 4->1

Reduced Matrix
for Node 14 (1, 4)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	0
3	0	0	∞	∞	12
4	∞	4	7	∞	0
5	0	11	∞	∞	

This is the Reduced Matrix
for Node 142
For edge (4, 2)

Node 142: For edge (4, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	0
3	0	∞	∞	∞	12
4	∞	∞	∞	∞	∞
5	1	∞	0	∞	∞

Reduced Cost at Node 142 (4, 2)
= $11 + 4 + 30 = 45$

This is the Reduced Matrix
for Node 143
For edge (4, 3)

Node 143: For edge (4, 3)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	∞	0
3	∞	0	∞	∞	12
4	∞	∞	∞	∞	∞
5	5	0	∞	∞	∞

Reduced Cost at Node 143 (4, 3)
= $7 + 7 + 30 = 44$

This is the Reduced Matrix
for Node 145
For edge (4, 5)

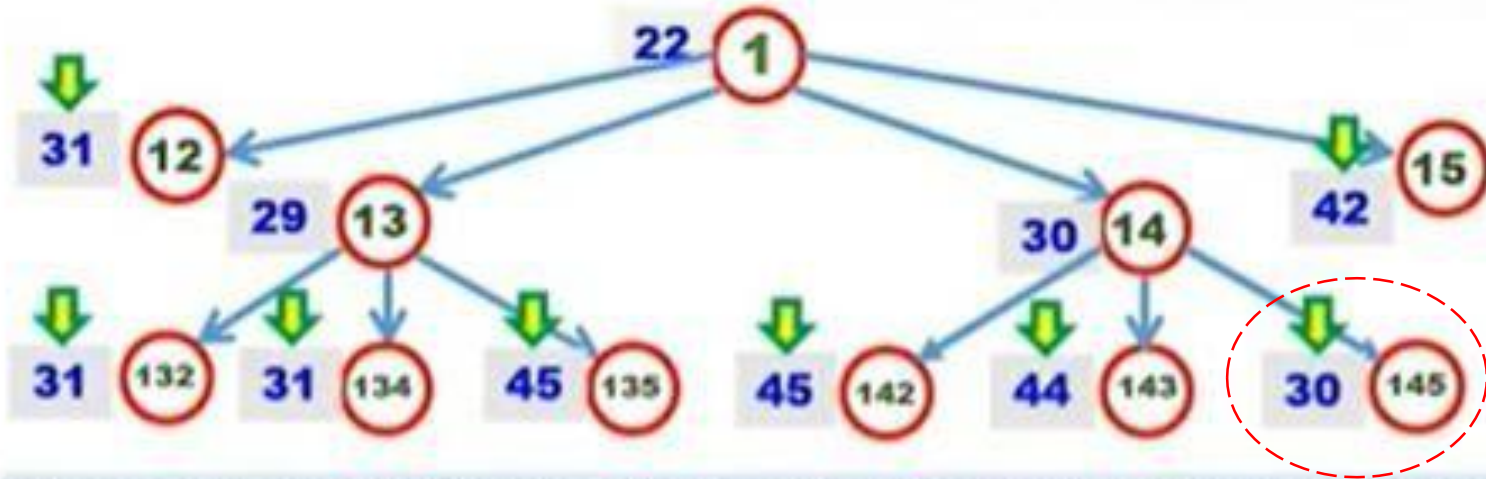
Node 145: For edge (4, 5)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	∞
3	0	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	0	11	∞	∞

Reduced Cost at Node 145 (4, 5)
= $0 + 0 + 30 = 30$

Reduced Cost = 30

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

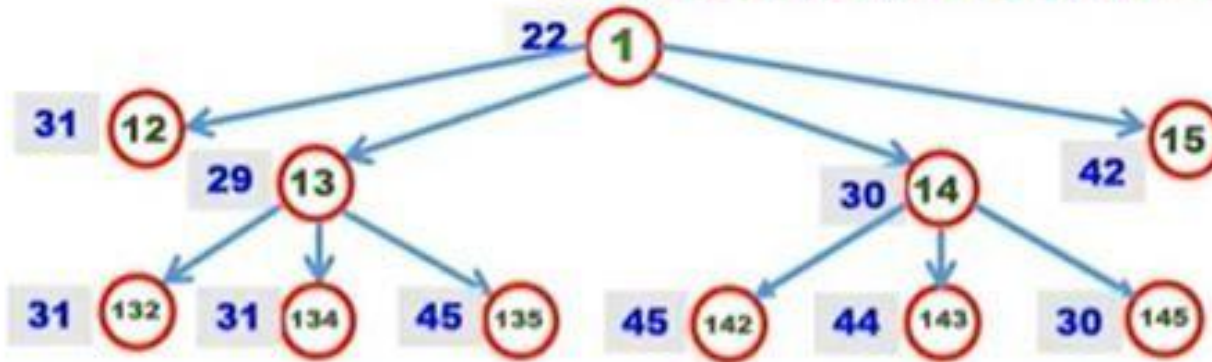


Among all the Leaf Node Values, find the Least Cost Node to explore and branch

Among the Leaf node Values 31, 31, 31, 45, 45, 44, 30 and 42, 30 is the Least Cost.

Therefore, we explore and branch from Node 145

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



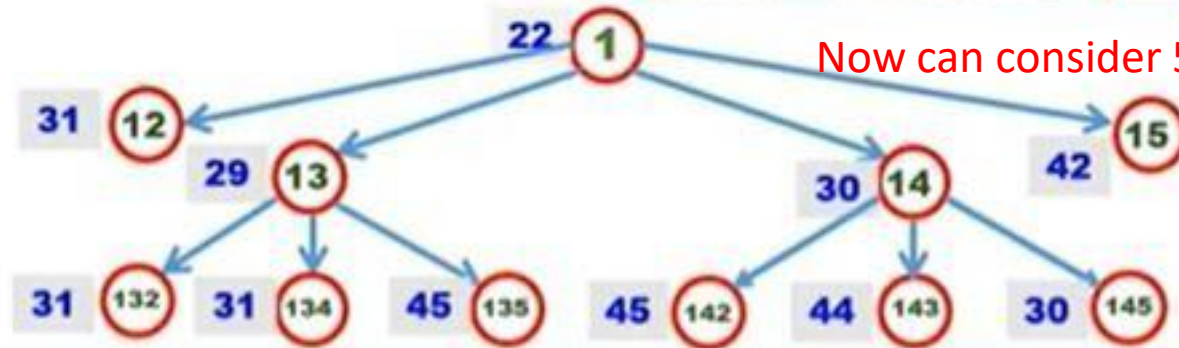
Reduced Matrix
for Node 145
(4, 5) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	∞
3	0	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	0	11	∞	∞

Reduced Cost = 30

Among all the Leaf Node Values, find the Least Cost Node to explore and branch
Among the Leaf node Values 31, 31, 31, 45, 45, 44, 30 and 42, 30 is the Least Cost.
Therefore, we explore and branch from Node 145

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



Now can consider 5->2, and 5->3

Reduced Matrix
for Node 145
(4, 5)



	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	∞
3	0	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	0	11	∞	∞

Reduced Cost = 30

This is the Reduced Matrix
for Node 1452
For edge (5, 2)

This is the Reduced Matrix
for Node 1453
For edge (5, 3)

Node 1452: For edge (5, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	∞
3	0	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

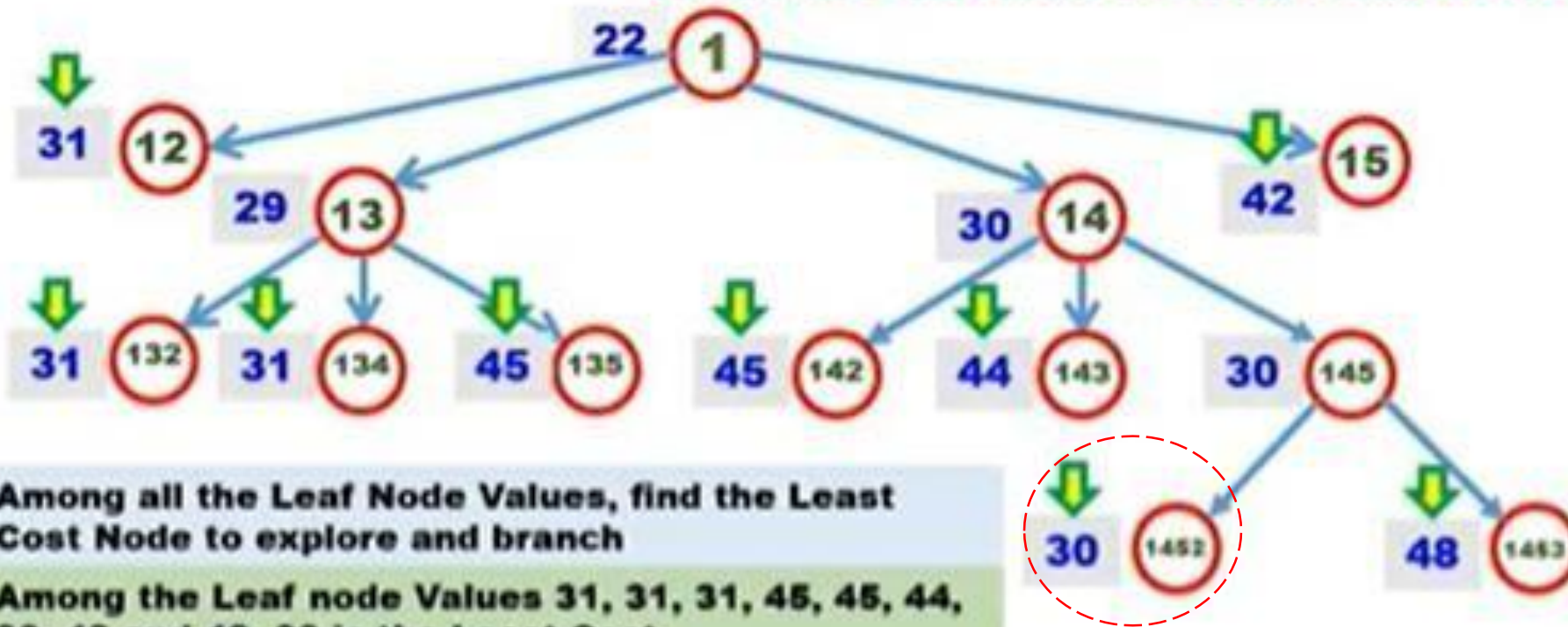
Reduced Cost at Node 1452 (5, 2)
= 0 + 0 + 30 = 30

Node 1453: For edge (5, 3)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	∞	∞
3	∞	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Reduced Cost at Node 1453 (5, 3)
= 7 + 11 + 30 = 48

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

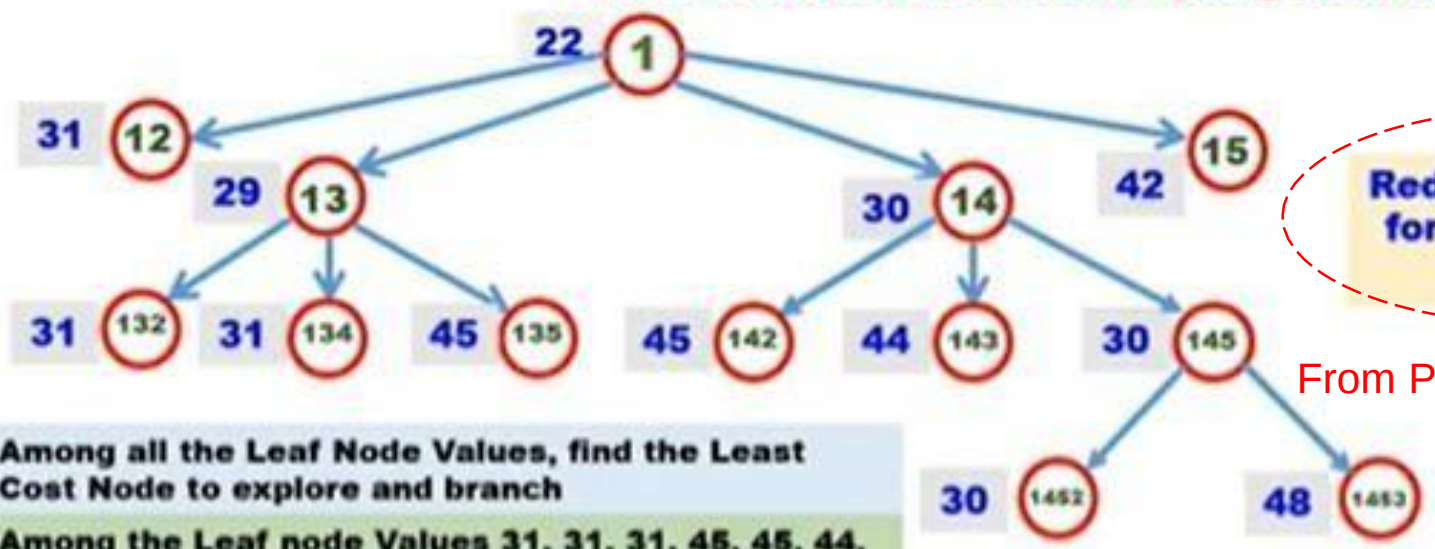


Among all the Leaf Node Values, find the Least Cost Node to explore and branch

Among the Leaf node Values 31, 31, 31, 45, 45, 44, 30, 48 and 42, 30 is the Least Cost.

Therefore, we explore and branch from Node 1452

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



Reduced Matrix
for Node 1452
(5, 2) →

From Previous Calculations

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	∞
3	0	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

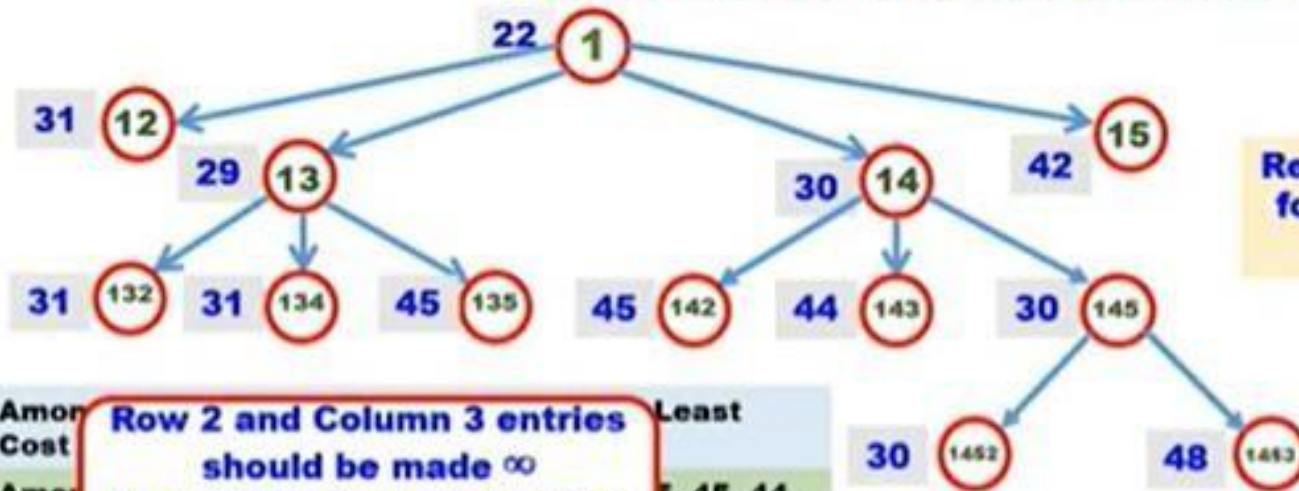
Reduced Cost = 30

Among all the Leaf Node Values, find the Least Cost Node to explore and branch

Among the Leaf node Values 31, 31, 31, 45, 45, 44, 30, 48 and 42, 30 is the Least Cost.

Therefore, we explore and branch from Node 1452

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



Reduced Matrix
for Node 1452
(5, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	∞
3	0	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Reduced Cost = 30

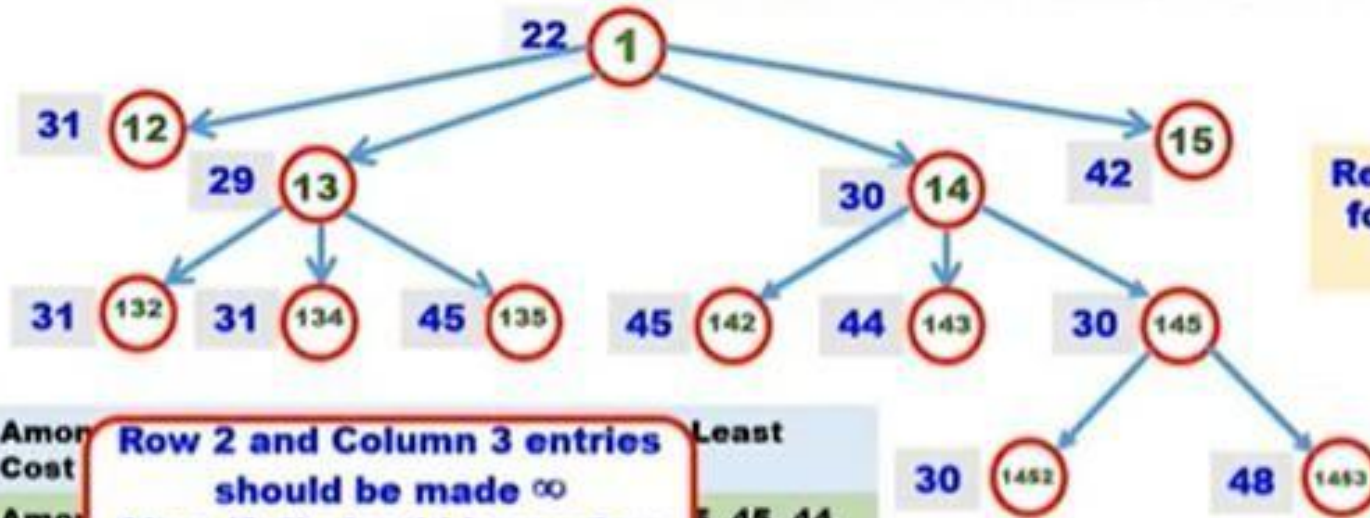
Among Cost
Among 30, 4
Therefore, Least
Cost 3, 45, 44,
Also, (3,1) should be made ∞

Node 14523: For edge (2, 3)

	1	2	3	4	5
1	∞		∞		
2	∞	∞	∞	∞	∞
3	∞		∞		
4			∞	∞	
5			∞		∞

Can not go from 3 to 1, hence (3,1) made infinity

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



Reduced Matrix
for Node 1452
(5, 2) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	∞
3	0	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Reduced Cost = 30

Among Cost
Among Cost
Therefor

Row 2 and Column 3 entries
should be made ∞
Also, (3,1) should be made ∞

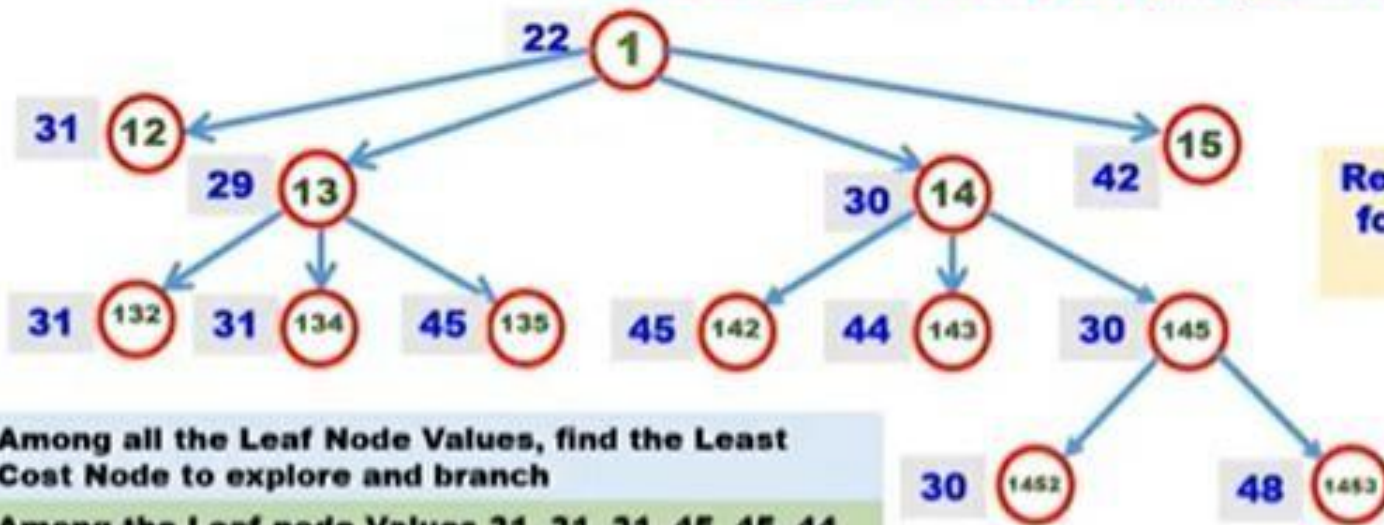
Least
5, 45, 44,
Node 1452

Node 14523: For edge (2, 3)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	∞	∞	∞
3	∞	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Copy remaining values are copied from reduced matrix after inserting inf

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



Reduced Matrix
for Node 1452
(5, 2) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	∞
3	0	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Reduced Cost = 30

Among all the Leaf Node Values, find the Least Cost Node to explore and branch

Among the Leaf node Values 31, 31, 31, 45, 45, 44, 30, 48 and 42, 30 is the Least Cost.

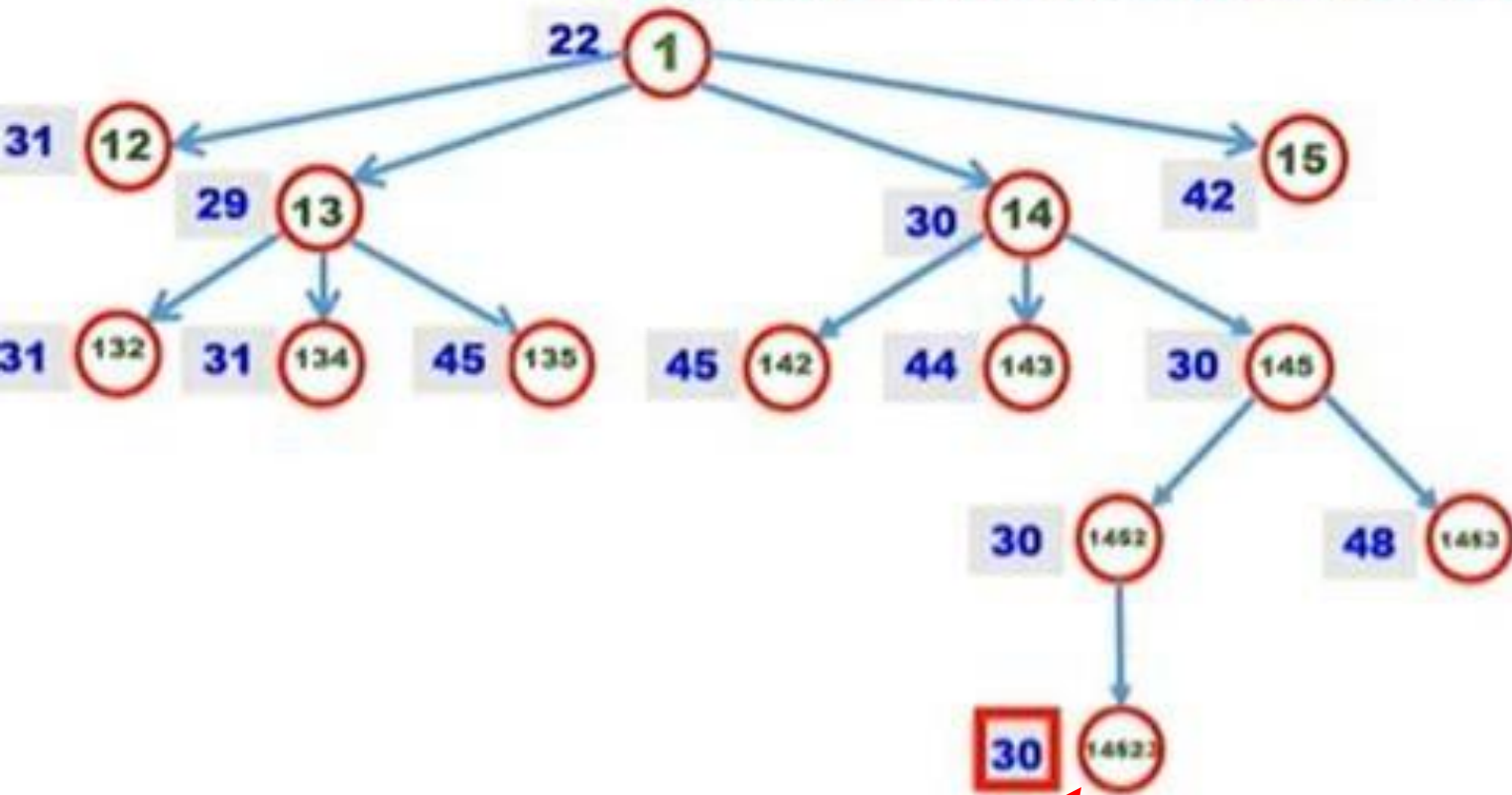
Therefore, we explore and branch from Node 1452

Node 14523: For edge (2, 3)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	∞	∞	∞
3	∞	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Reduced Cost at Node 14523 (2, 3) = 0 + 0 + 30 =

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



The Cost Matrix for the Graph is as follows:

	1	2	3	4	5
1	∞	14	4	10	20
2	14	∞	7	8	7
3	4	5	∞	4	16
4	11	7	9	∞	2
5	18	7	17	4	∞

Cost taken from original adjacency matrix

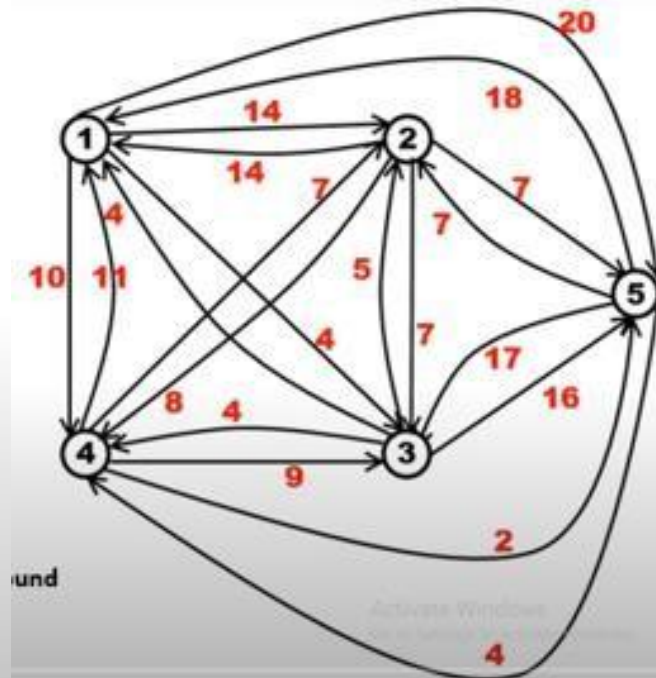
Among the Leaf node Values 31, 31, 31, 45, 45, 44, 30, 48 and 42, 30 is the Least Cost.

We have got the solution as 14523 and again back to 1. The Solution is as follows:

1 → 4 → 5 → 2 → 3 → 1

FINAL COST = 10 + 2 + 7 + 7 + 4 = 30

Consider the following Graph:



Ref

Refer Video : Travelling Salesman Problem using Branch and Bound

<https://www.youtube.com/watch?v=HjSbaKF8Gi0>

- <https://www.javatpoint.com/branch-and-bound>
- <https://www.geeksforgeeks.org/introduction-to-branch-and-bound-data-structures-and-algorithms-tutorial/>